

Inteligență Artificială

Bogdan Alexe

bogdan.alexe@fmi.unibuc.ro

Radu Ionescu

raducu.ionescu@gmail.com

Informatică, anul II, 2025-2026

Recapitulare – cursul trecut

1. Agenți inteligenți și mediile în care aceștia funcționează
2. Rezolvarea problemelor prin căutare
 - Formularea unei probleme
 - Graful stărilor
 - Arborele de căutare
 - Căutare într-un arbore de căutare
3. Căutare neinformată
 - Căutare în lățime (Bread-First Search)
 - Căutare în adâncime (Depth-First Search)
 - Căutare în adâncime limitată (Depth-Limited Search)
 - Căutare iterativă în adâncime (Iterative Deepening Search)
 - Căutare uniformă după cost (Uniform-Cost Search)
 - Căutare bidirecțională (Bidirectional Search)

Recapitulare: cursul trecut

Probleme definite de:

- stare inițială, stare/stări finale, acțiuni, costul unei acțiuni, funcție succesor, funcție cost

Soluția optimă:

- are funcția cost cea mai mica dintre toate soluțiile

Căutare:

- găsește soluția explorând arborele de căutare
- nod = (stare, părinte, acțiune, cost)
- arce = acțiuni

Cuprinsul cursului de azi

1. Căutare neinformată

- Căutare în lăţime (Breadth-First Search)
- Căutare în adâncime (Depth-First Search)
- Căutare în adâncime limitată (Depth-Limited Search)
- Căutare ierativă în adâncime (Iterative Deepening Search)
- Căutare uniformă după cost (Uniform-Cost Search)
- Căutare bidirecţională (Bidirectional Search)

2. Căutare informată

- Funcţii euristice
- Căutare Greedy (Greedy Best-First Search)

Algoritmul de căutare în arbore (TREE-SEARCH)

funcția TREE-SEARCH (**problema**, **strategie**) **returnează** o soluție sau eșec

inițializează **frontiera** folosind **starea inițială a problemei**

ciclează

dacă frontiera este vidă **atunci returnează** eșec

alege un **nod frunză** conform **strategiei** și elimină-l din **frontieră**

dacă nodul conține o **stare scop**

atunci returnează soluția corespunzătoare

expandează nodul ales, adăugând nodurile succesori generate în **frontieră**

Frontiera = structură de date care conține nodurile generate, dar încă neexpandate

- coadă (FIFO) – se extrage cel mai vechi nod din frontier (căutare în lățime)
- stivă (LIFO) – se extrage cel mai recent nod adăugat în frontier (căutare în adâncime)
- coadă de priorități – se extrage nodul cu prioritatea cea mai mare conform unei funcții de evaluare / ordonare (căutare uniformă după cost)

Strategii de căutare

Strategie = alegerea unei ordini de explorare a nodurilor

Strategiile de căutare se evaluează conform următoarelor patru criterii:

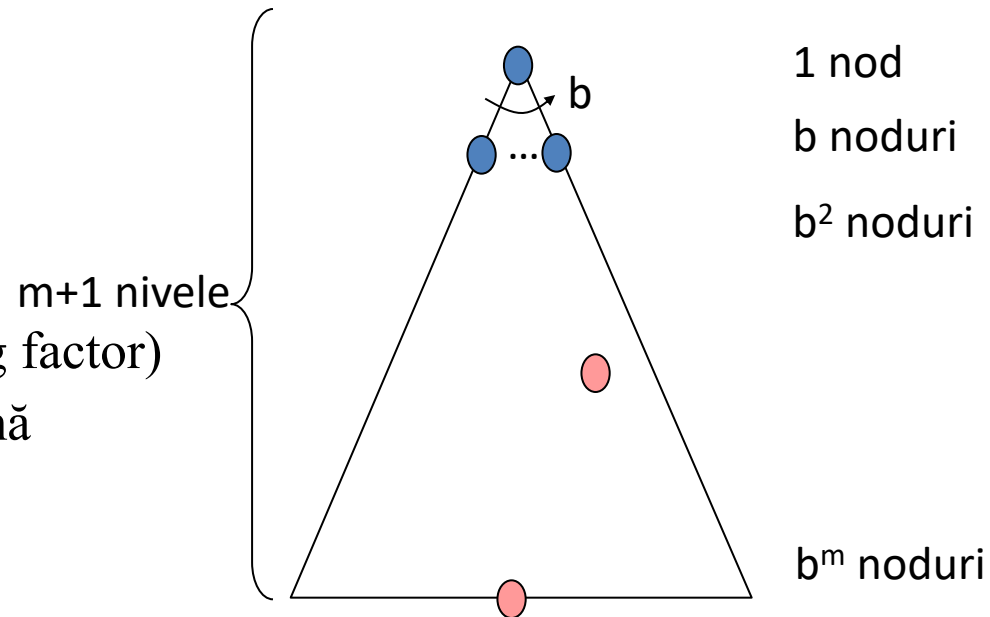
1. **Completitudine**: dacă, atunci când o soluție există, strategia dată garantează găsirea acesteia
2. **Complexitate a timpului**: durata în timp (număr de noduri generate) pentru găsirea unei soluții
3. **Complexitate a spațiului**: necesitățile de memorie (pentru a memora numărul maxim de noduri necesare) pentru efectuarea căutării
4. **Optimalitate**: atunci când există mai multe soluții, strategia dată o găsește pe cea mai bună dintre ele (pe baza unei funcții – spre exemplu găsește funcția cu costul cel mai mic)

Strategii de căutare

- Completitudine: garantarea unei soluții dacă ea există
- Optimalitate: garantarea unei soluții de cost minim
- Complexitate timp
- Complexitatea spațiu

- Ilustrare arbore de căutare:
 - b = factor ramificare (branching factor)
 - m = adâncimea (nivelul) maximă
 - soluții la adâncimi diferite

- Numărul de noduri în arbore?
 - $1 + b + b^2 + \dots + b^m = O(b^m)$

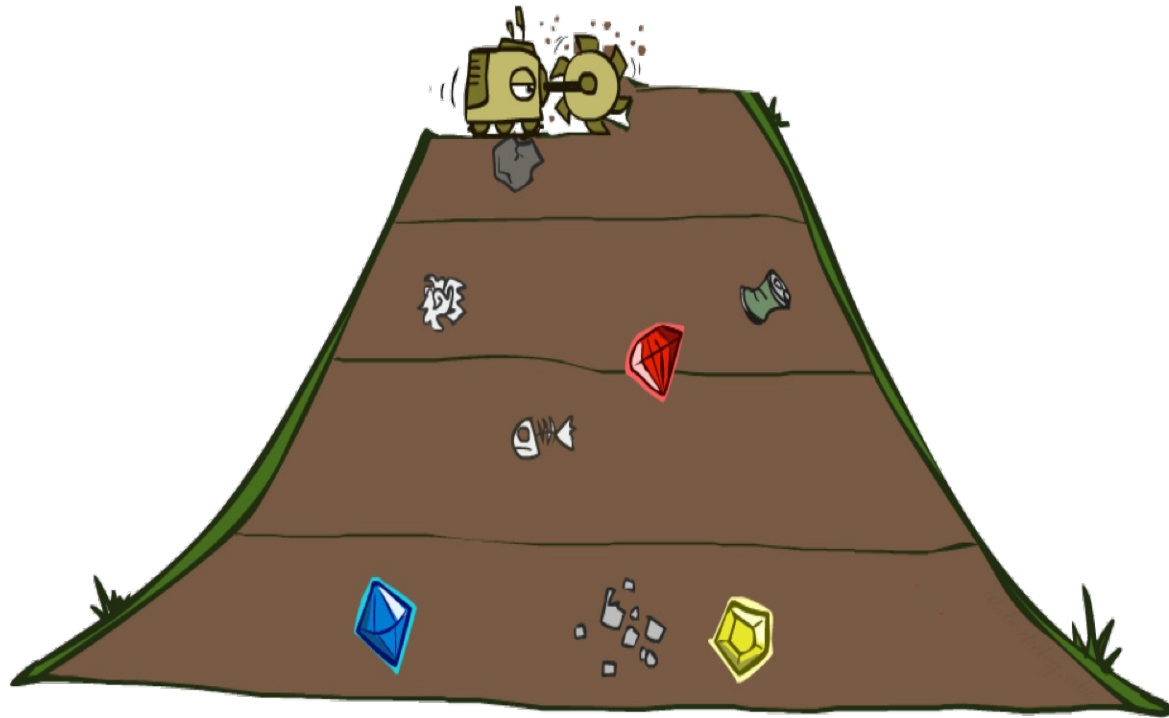


Strategii de căutare neinformată

Strategii neinformate – folosesc numai informație disponibilă din definirea problemei. Pot genera succesori și distinge dacă o stare este scop sau. Din acest motiv se mai numesc și strategii de căutare *oare*.

- căutare în lățime (breadth-first search)
- căutare în adâncime (depth-first search)
- căutare cu cost uniform (uniform-cost search)
- căutare cu adâncime limitată (depth-limited search)
- căutare cu adâncime incrementală (iterative deepening search)
- căutare bidirecțională (bidirectional search)

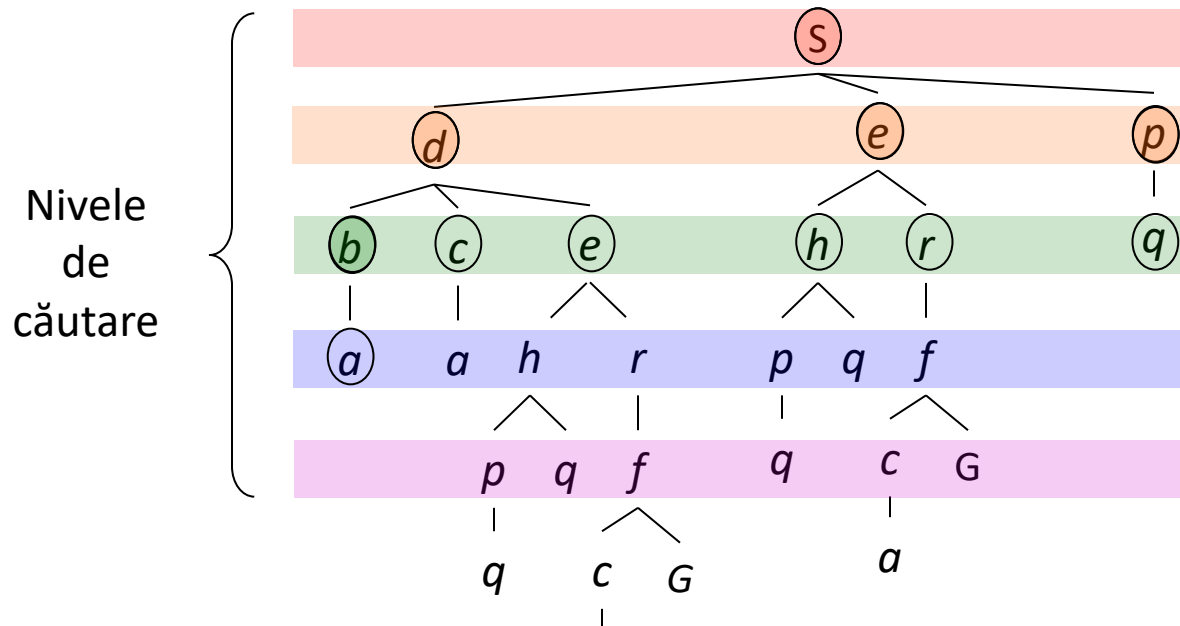
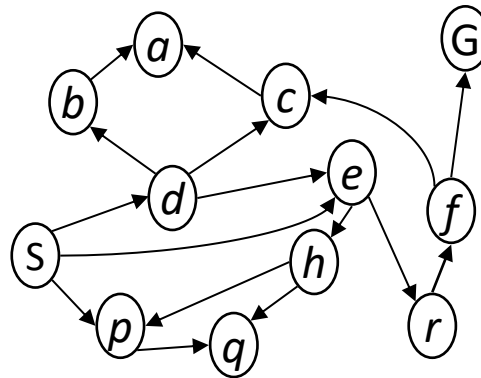
Căutare în lățime (breadth-first = BF)



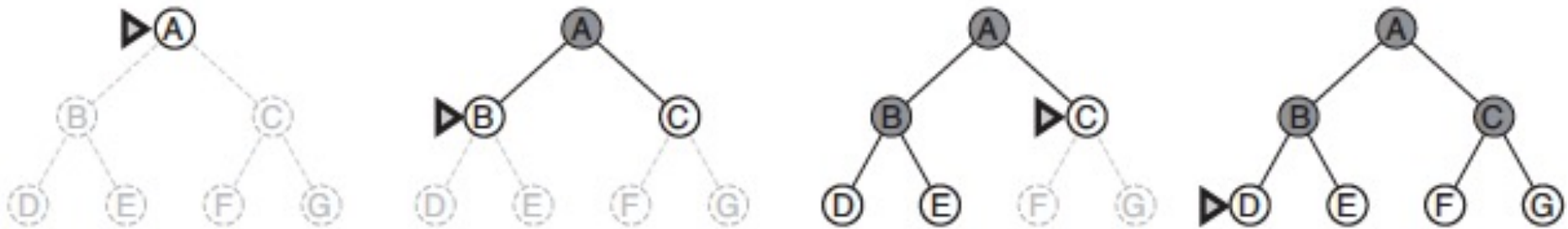
Căutare în lăţime (breadth-first = BF)

Strategie: expandează succesiv toate nodurile în ordinea adâncimii începând cu nodul rădăcină

Frontiera este implementată cu o coadă (FIFO)



Căutare în lățime pe un arbore binar



Căutare în lățime pe un arbore binar: la fiecare pas, săgeata indică nodul care va fi expandat.

Proprietățile căutării în lățime

- Ce fel de noduri expandează BF?

- procesează toate nodurile deasupra soluției de adâncime minimă
- fie s = adâncimea minimă a unui nod scop
- complexitate timp $O(b^s)$
- verifică dacă un nod succesori generat este nod scop - reduce complexitatea timp de la $O(b^{s+1})$ la $O(b^s)$

- Cât de mult spațiu necesită BF?

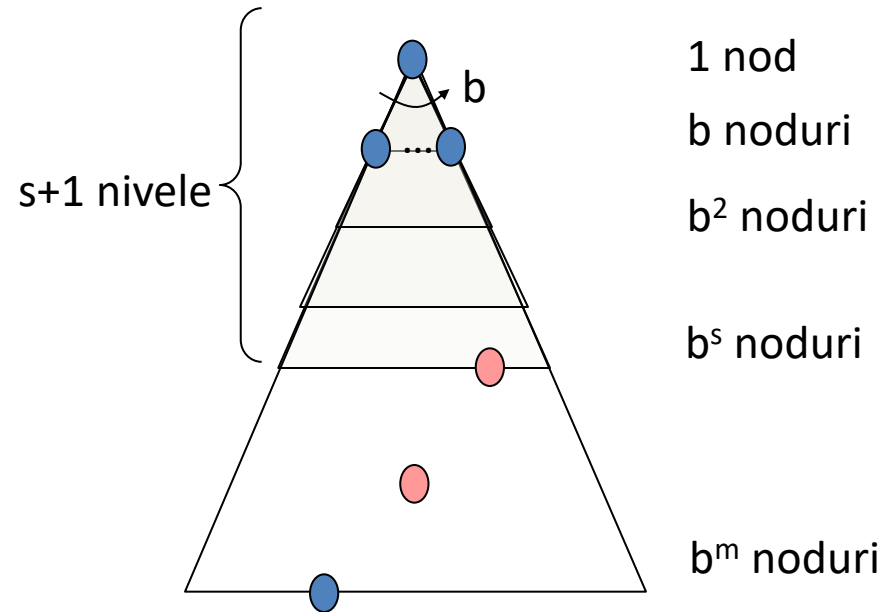
- este spațiul pentru memorarea frontierei $O(b^s)$ + memorarea nodurilor explorate $O(b^{s-1})$
- limită superioară dată de ultimul nivel
- complexitate $O(b^s)$

- Completitudine?

- dacă o soluție există atunci s este finit, deci DA!

- Optimalitate?

- în general NU!
- uneori DA (spre exemplu dacă costurile fiecărei muchii sunt egale)



Timpul și memoria necesară pentru căutarea în lățime - BF

Considerăm:

- factor de ramificare (branching factor) $b = 10$;
- timp de procesare = 1 milion de noduri/secundă;
- necesar de memorie pentru stocarea unui nod = 1000 bytes

Adâncime	#Noduri	Timp	Memorie
2	110	.11 ms	107 kB
4	11110	11 ms	10.6 MB
6	10^6	1.1 sec	1 GB
8	10^8	2 min	103 GB
10	10^{10}	3 ore	10 TB
12	10^{12}	13 zile	1 PB
14	10^{14}	3.5 ani	99 PB
16	10^{16}	350 ani	10 EB

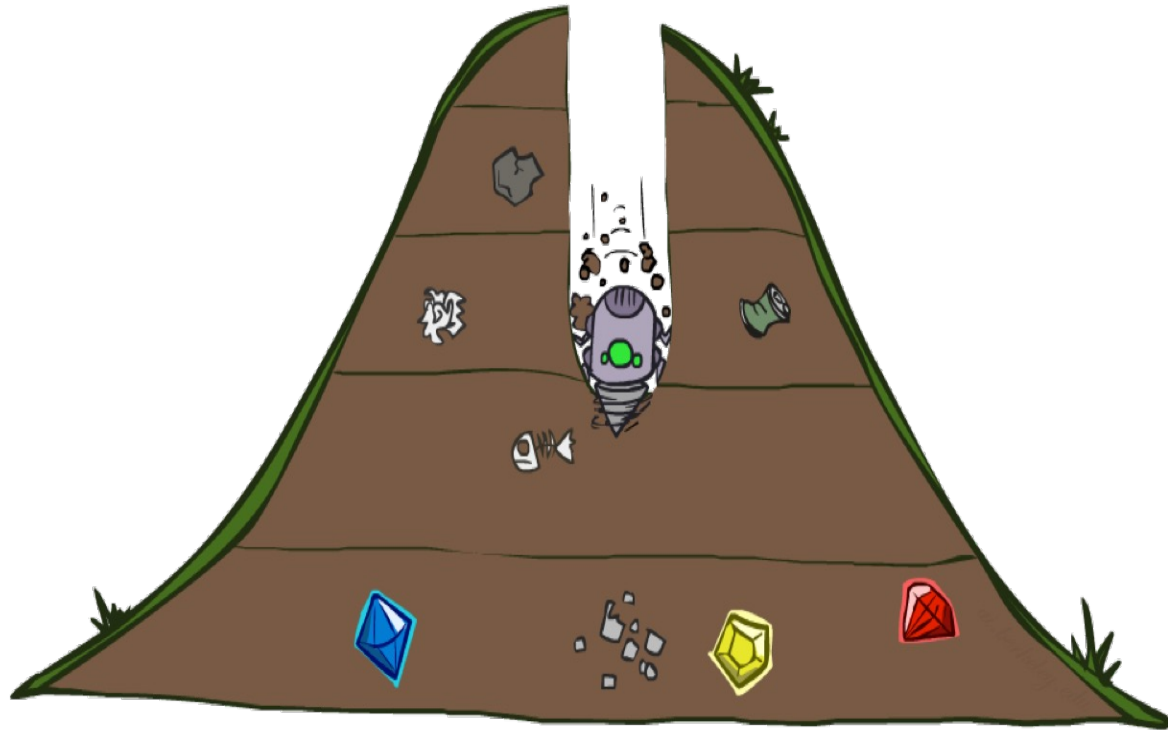
Timpul și memoria necesară pentru căutarea în lățime - BF

Adâncime	#Noduri	Timp	Memorie
2	110	.11 ms	107 kB
4	11110	11 ms	10.6 MB
6	10^6	1.1 sec	1 GB
8	10^8	2 min	103 GB
10	10^{10}	3 ore	10 TB
12	10^{12}	13 zile	1 PB
14	10^{14}	3.5 ani	99 PB
16	10^{16}	350 ani	10 EB

Remarci:

- complexitate exponențială timp și spațiu
- timp de execuție aproape infinit pentru adâncimi relativ moderate
- memoria e un factor de limitare mai mare decât timpul

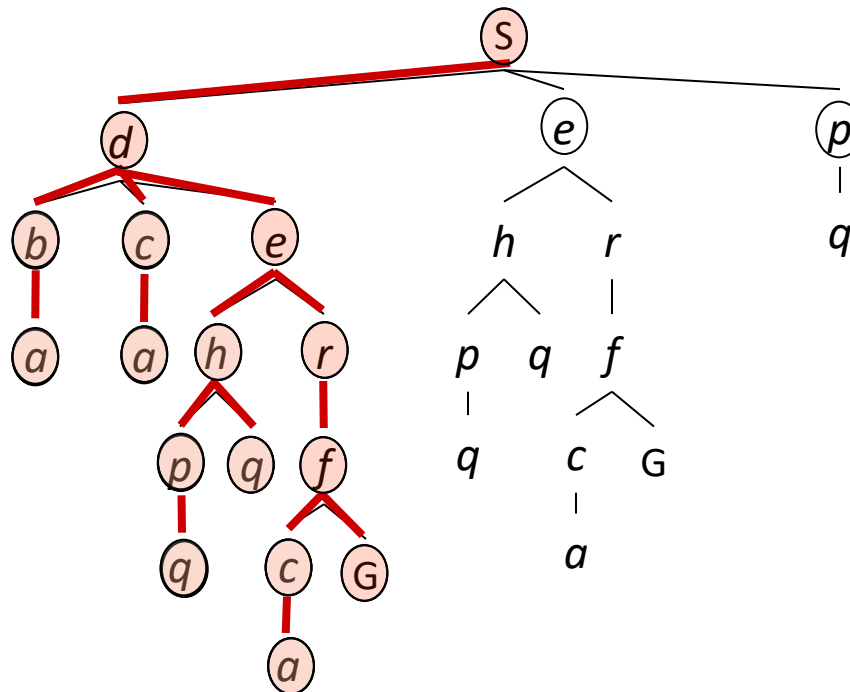
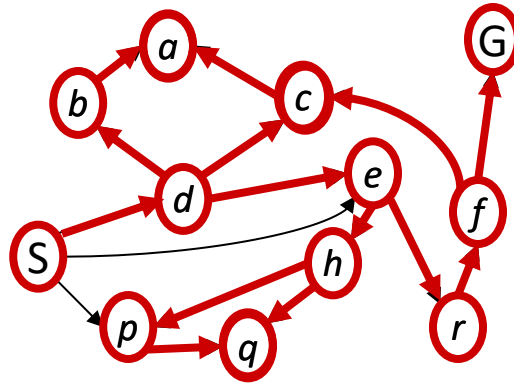
Căutarea în adâncime (depth-first = DF)



Căutarea în adâncime (depth-first = DF)

Strategie: expandează
nodul curent de
adâncime maximă

Frontiera este
implementată cu o
stivă (LIFO)



Proprietățile căutării în adâncime

- Ce fel de noduri expandează DF?

- expandează partea stângă prefix din arbore (cel mai adânc nod neexpandat din frontieră)
- urmează o ramură până la capăt apoi revine (backtracking)
- poate explora întreg arborele!
- complexitate timp $O(b^m)$

- Cât de mult spațiu necesită DF?

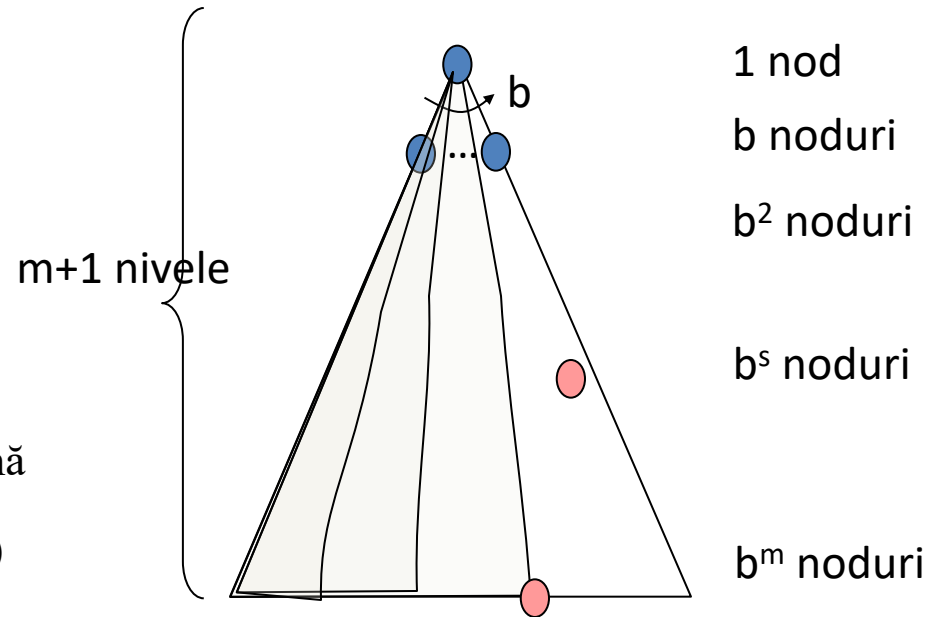
- memorează drumul curent de la rădăcină la nodul curent + frații neexplorați ai nodurilor de pe acest drum, deci $O(bm)$

- Completitudine?

- m ar putea fi infinit, în acest caz NU!
- m finit și dacă se memorează stările vizitate (pentru evitarea ciclurilor) DA!

- Optimalitate?

- NU, găsește “cea mai din stânga” soluție, indiferent de adâncime sau cost

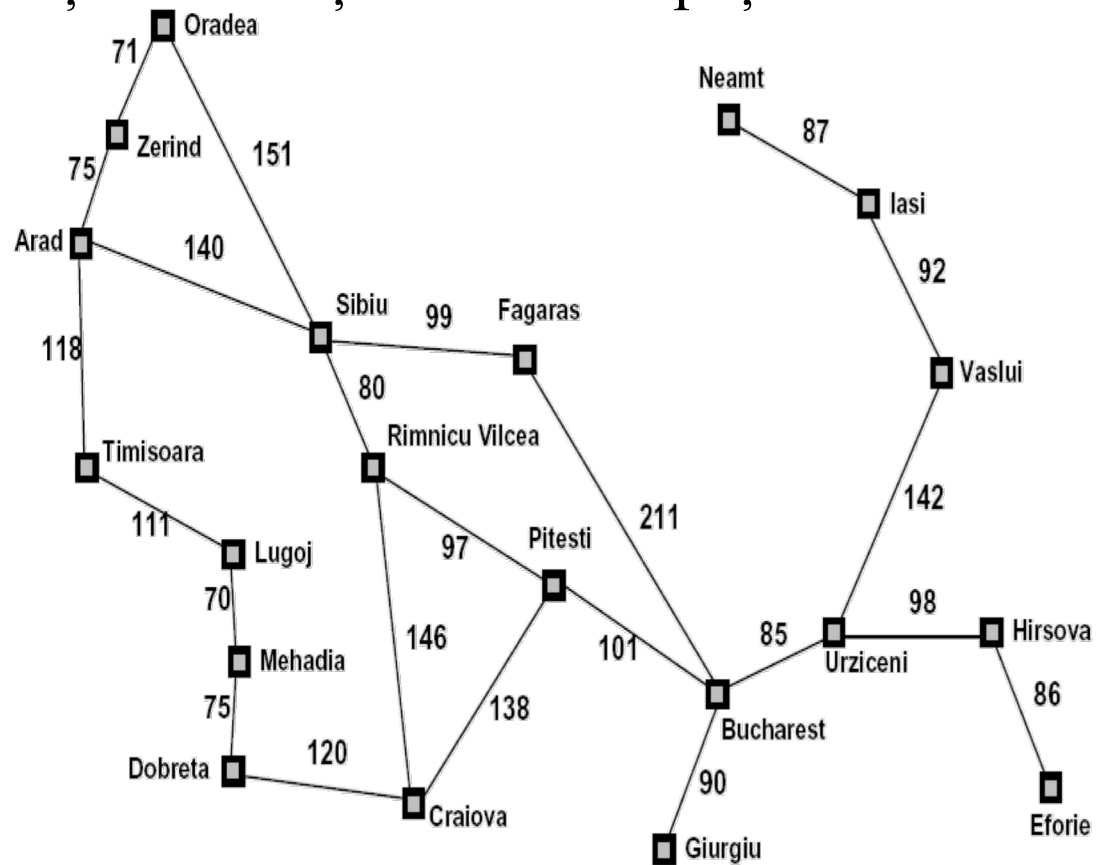


Căutare în adâncime limitată

- căutarea în adâncime poate eșua pentru spațiu de stări infinit
 - harta are 20 de orașe (stări)
 - adâncimea maximă poate fi considerată 19
 - pot ajunge din orice oraș în alt oraș în maxim 9 pași
 - diametru = 9
 - limitează DF la 9
-
- ```

graph TD
 Oradea[Oradea] ---|71| Sibiu[Sibiu]
 Zerind[Zerind] ---|75| Sibiu
 Oradea ---|151| Zerind
 Neamt[Neamt] ---|87| Sibiu

```



# Căutare în adâncime limitată

Căutare în adâncime până la un anumit nivel  $l$  (depth-limited DL).

- **Ce fel de noduri expandează DL?**

- expandează partea stângă din arbore (cel mai adânc nod neexpandat din frontieră), dar numai până la nivelul  $l$
- explorează o ramură până la limita  $l$ , apoi face backtracking
- complexitate timp  $O(b^l)$

- **Cât de mult spațiu necesită DL?**

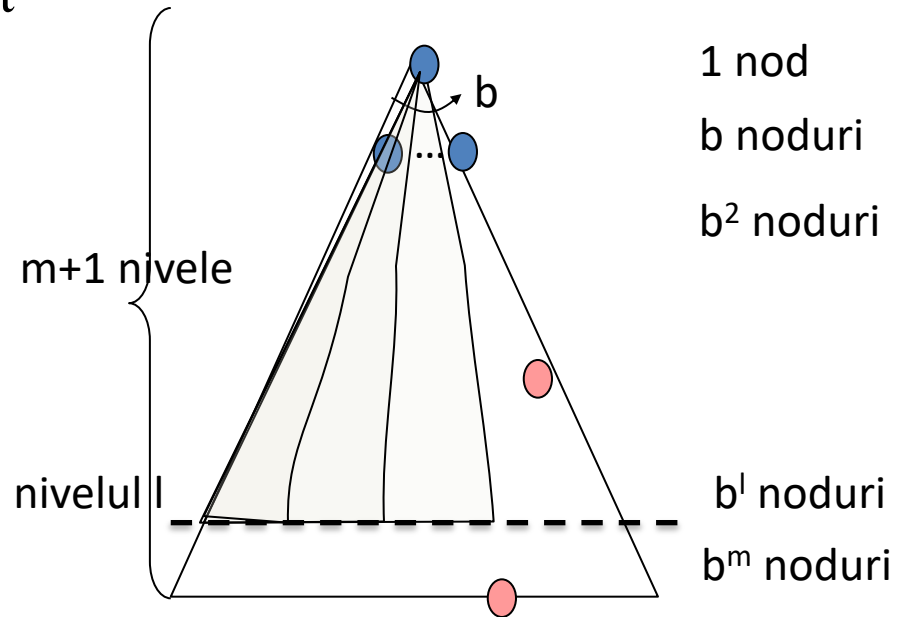
- memorează drumul curent de la rădăcină la nodul curent + frații neexplorați ai nodurilor de pe acest drum, deci  $O(bl)$

- **Completitudine?**

- NU, poate rata soluția dacă  $l$  este prea mic

- **Optimalitate?**

- NU, găsește “cea mai din stânga” soluție de adâncime maximă  $l$ , indiferent de cost



# Căutare iterativă în adâncime

- Combină căutarea în adâncime (DF) cu căutarea în lățime (BF):

- Rulează DF cu adâncimea limitată la 1.

Dacă nu găsește o soluție ...

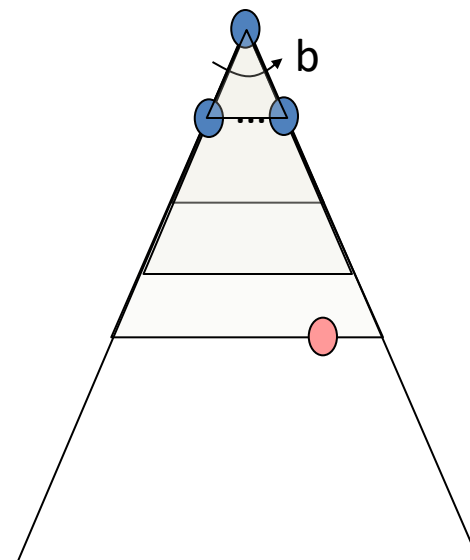
- Rulează DF cu adâncimea limitată la 2.

Dacă nu găsește o soluție ...

- Rulează DF cu adâncimea limitată la 3.

Dacă nu găsește o soluție ...

- Rulează DF cu adâncimea limitată la 4....



- iterative depth search = IDS

# Căutare iterativă în adâncime

## Ce fel de noduri expandează IDS?

DF limitat repetat  $\rightarrow$  produce explorare pe niveluri (ca BF)

Complexitate timp  $O(b^s)$

$N(\text{IDS}) = s*b + (s-1) * b^2 + \dots + b^s$  (noduri vizitate de IDS)

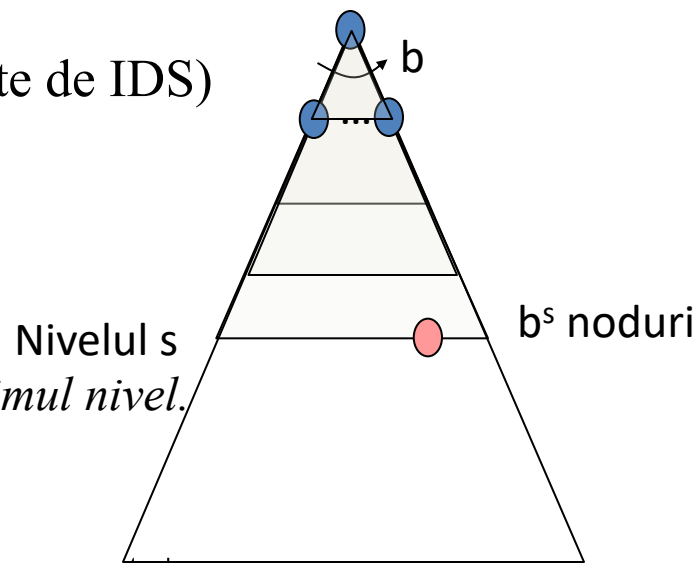
$N(\text{BF}) = b + b^2 + \dots + b^s$  (noduri vizitate de BF)

Pentru  $b = 10, s = 5$ :

$N(\text{IDS}) = 50 + 400 + \dots + 100000 = 123450$

$N(\text{BF}) = 10 + 100 + \dots + 100000 = 111110$

*supracostul este mic, majoritatea nodurilor sunt pe ultimul nivel.*



## Cât de mult spațiu necesită IDS? (de la DF)

memorează drumul curent de la rădăcină la nodul curent +  
frații neexplorați ai nodurilor de pe acest drum, deci  $O(bs)$

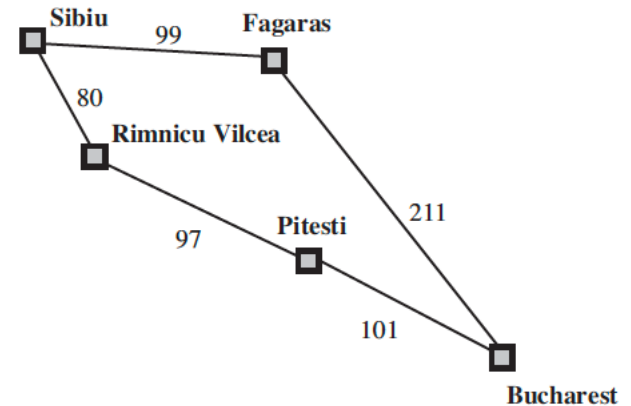
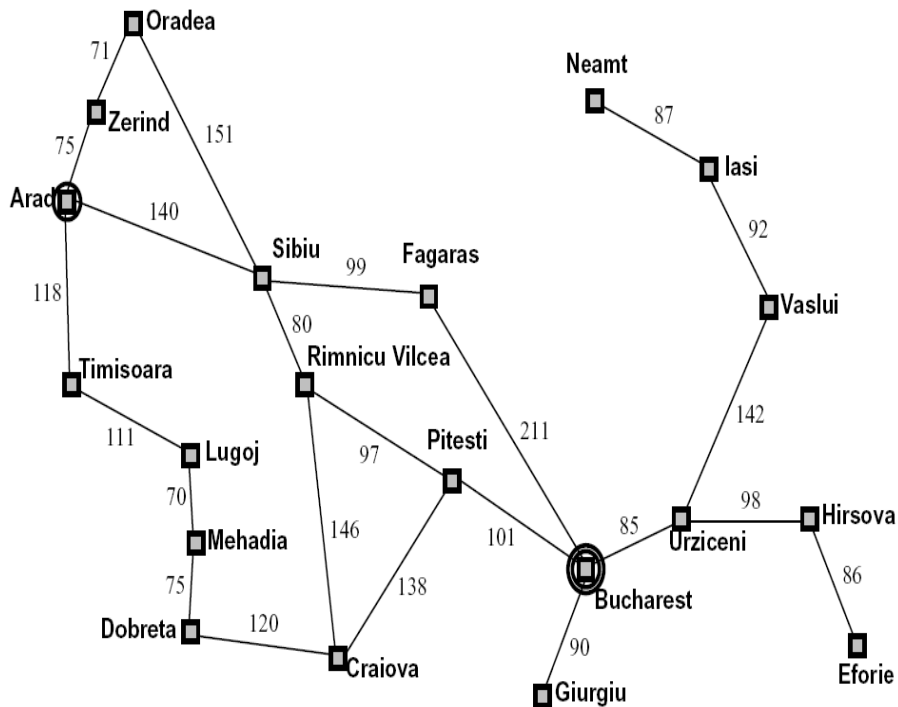
## Completitudine? (de la BF)

dacă o soluție există atunci s este finit, deci DA!

## Optimalitate? (de la BF)

dacă costurile fiecărei muchii sunt egale ( $= 1$ ) atunci DA!

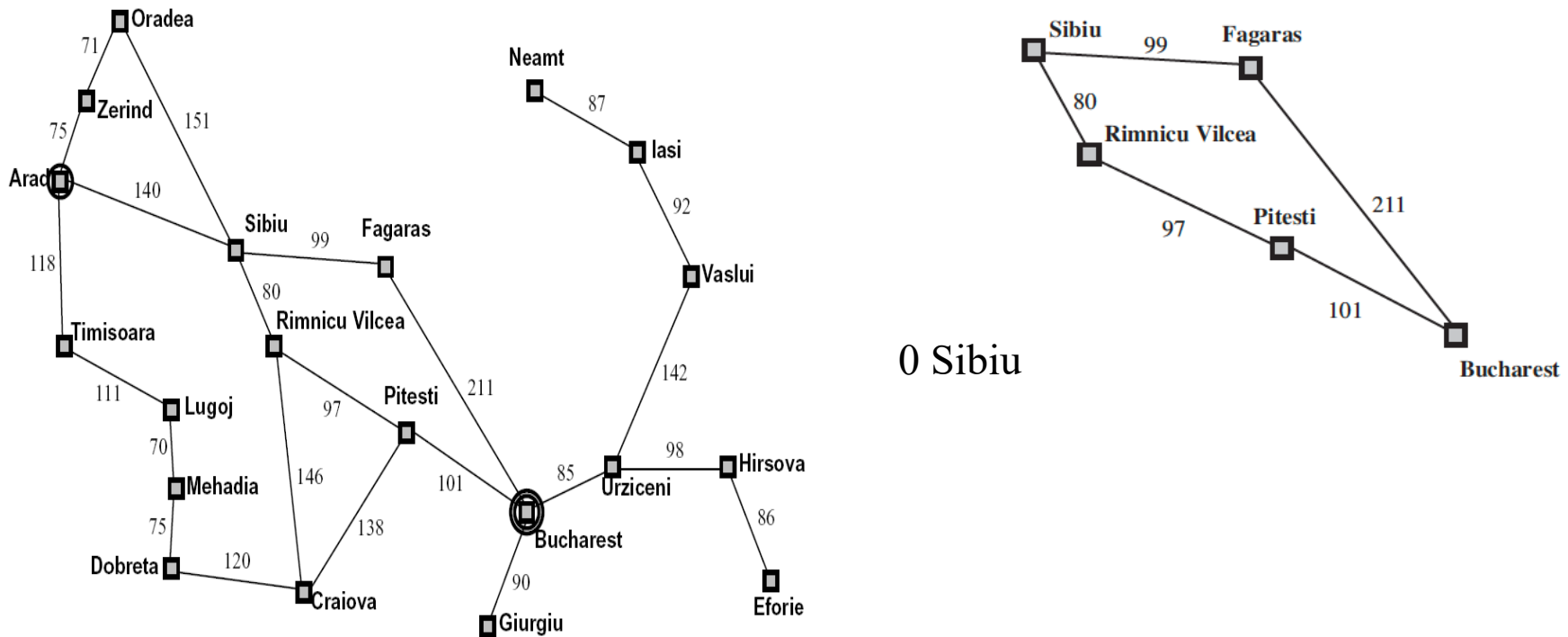
# Căutare uniformă după cost (uniform cost search – UCS)



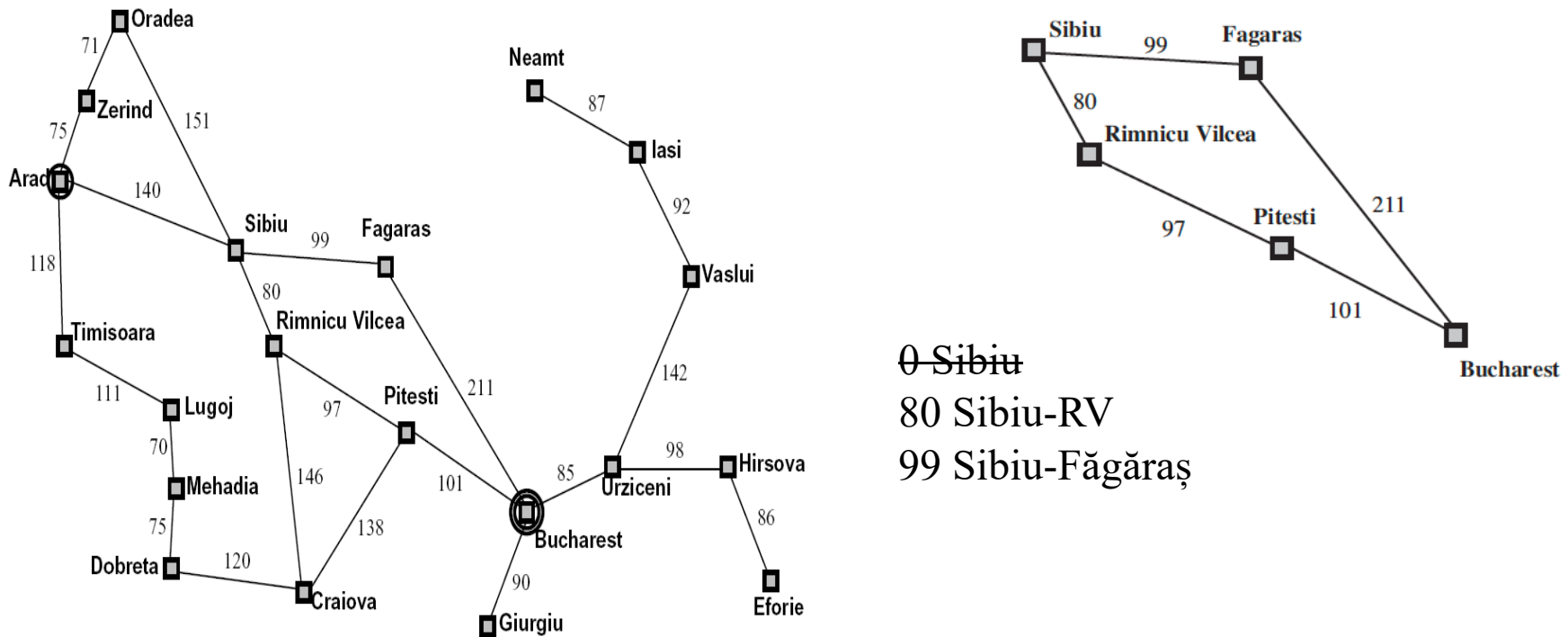
*Strategie: expandează nodul care face parte dintr-un drum de cost minim la pasul curent.*

*Frontiera este implementată cu o coadă de priorități (prioritate: costul minim al unui drum curent)*

# Căutare uniformă după cost (uniform cost search – UCS)

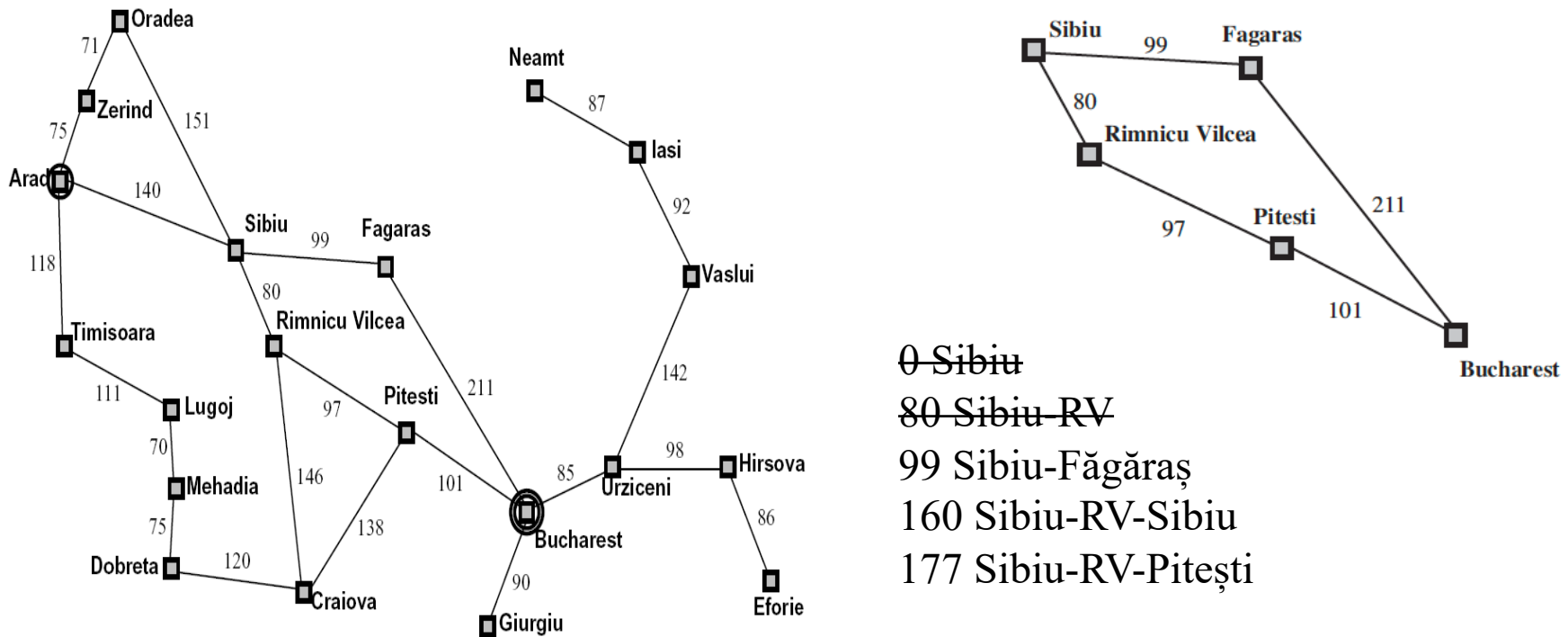


# Căutare uniformă după cost (uniform cost search – UCS)

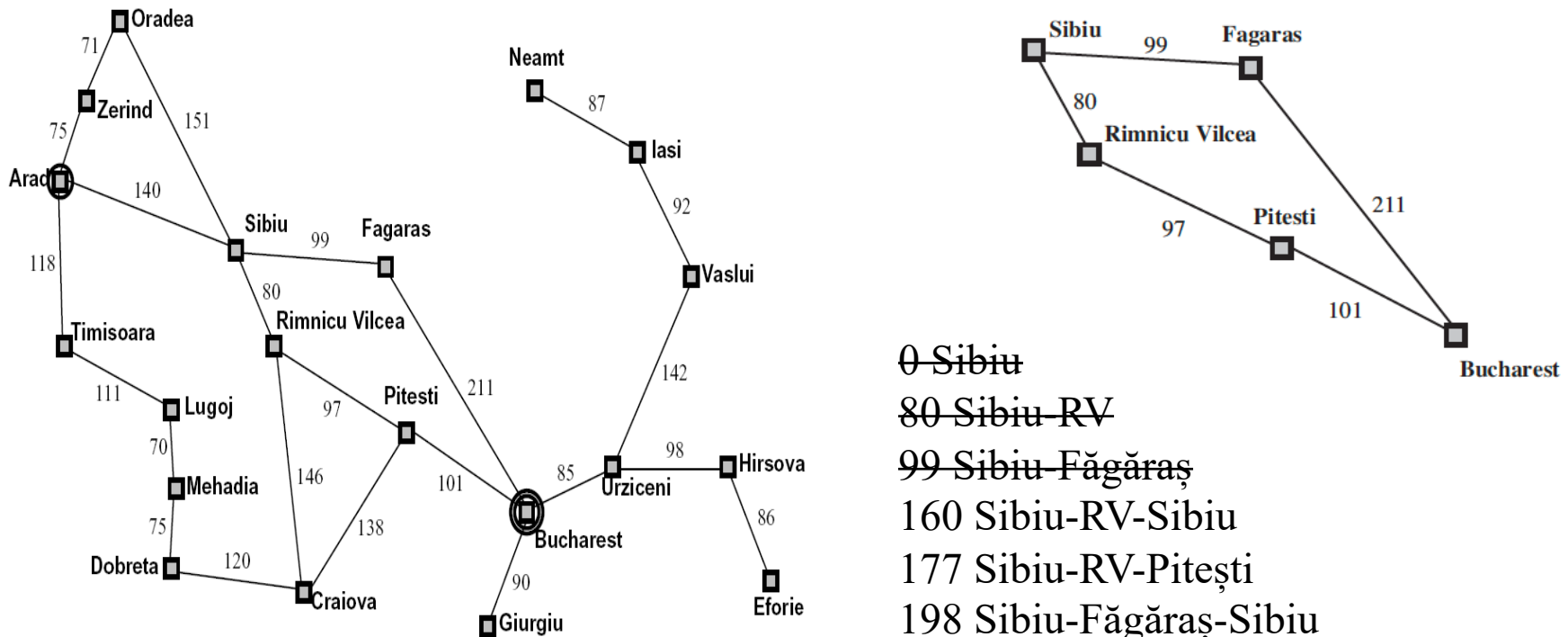




# Căutare uniformă după cost (uniform cost search – UCS)

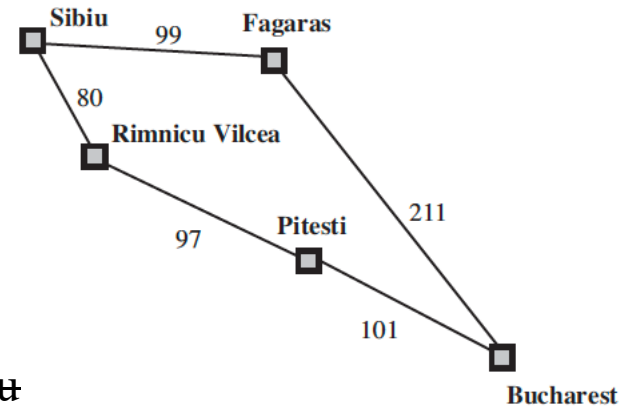
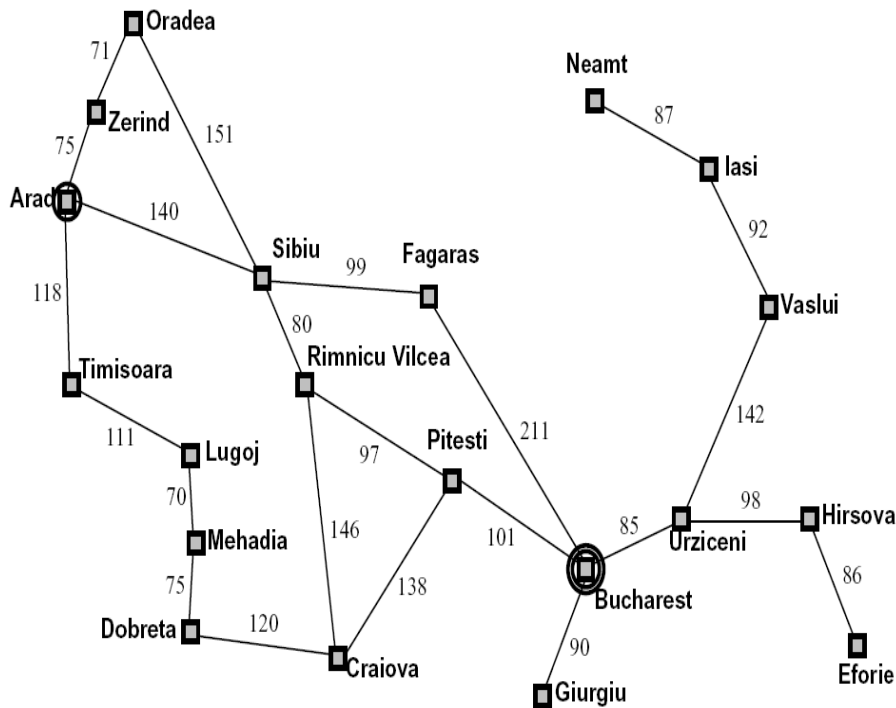


# Căutare uniformă după cost (uniform cost search – UCS)



~~0 Sibiu~~  
~~80 Sibiu-RV~~  
~~99 Sibiu-Făgăraș~~  
 160 Sibiu-RV-Sibiu  
 177 Sibiu-RV-Pitești  
 198 Sibiu-Făgăraș-Sibiu  
 310 Sibiu-Făgăraș-București

# Căutare uniformă după cost (uniform cost search – UCS)



~~0 Sibiu~~

~~80 Sibiu-RV~~

~~99 Sibiu-Făgăraș~~

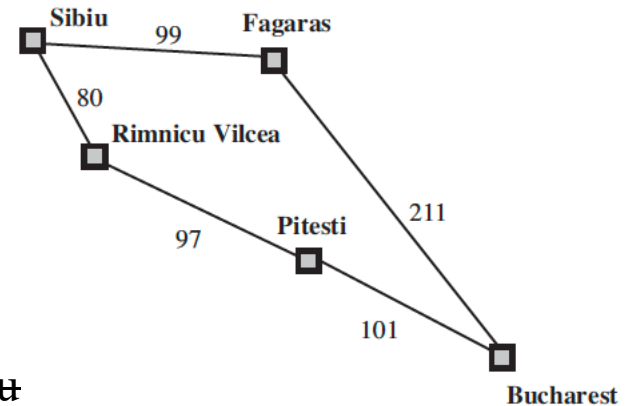
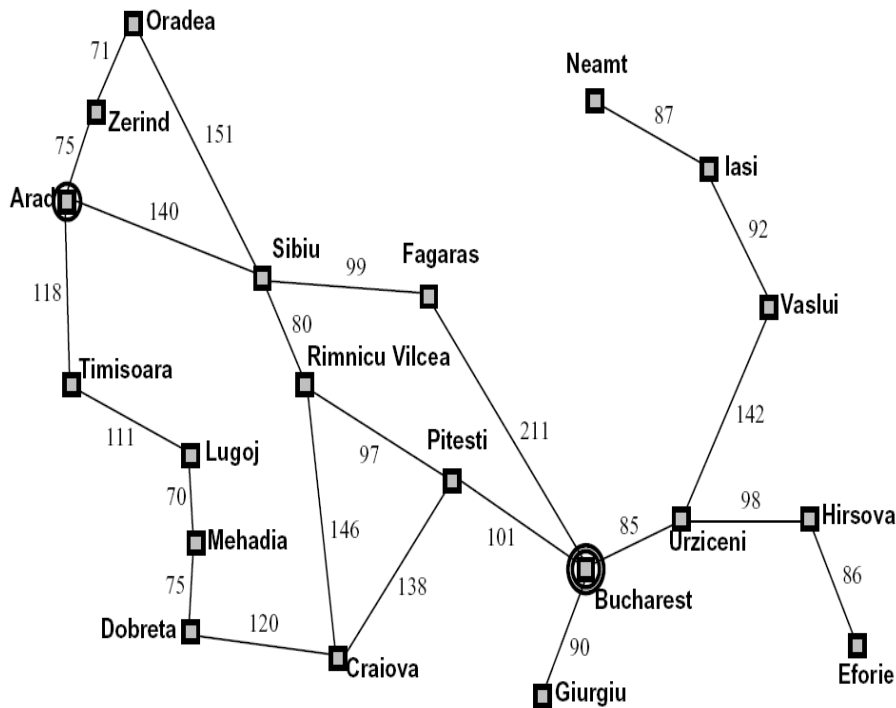
~~160 Sibiu-RV-Sibiu~~

~~177 Sibiu-RV-Pitești~~

~~198 Sibiu-Făgăraș-Sibiu~~

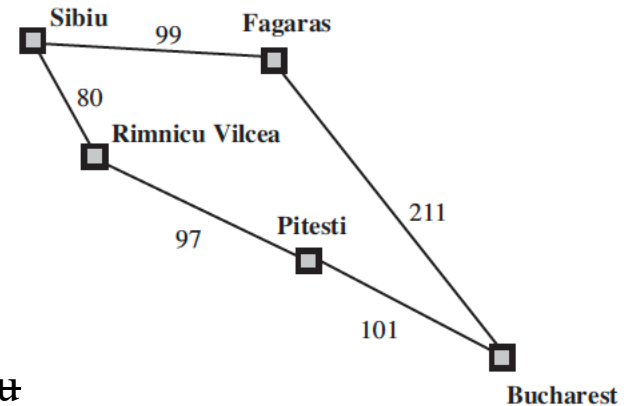
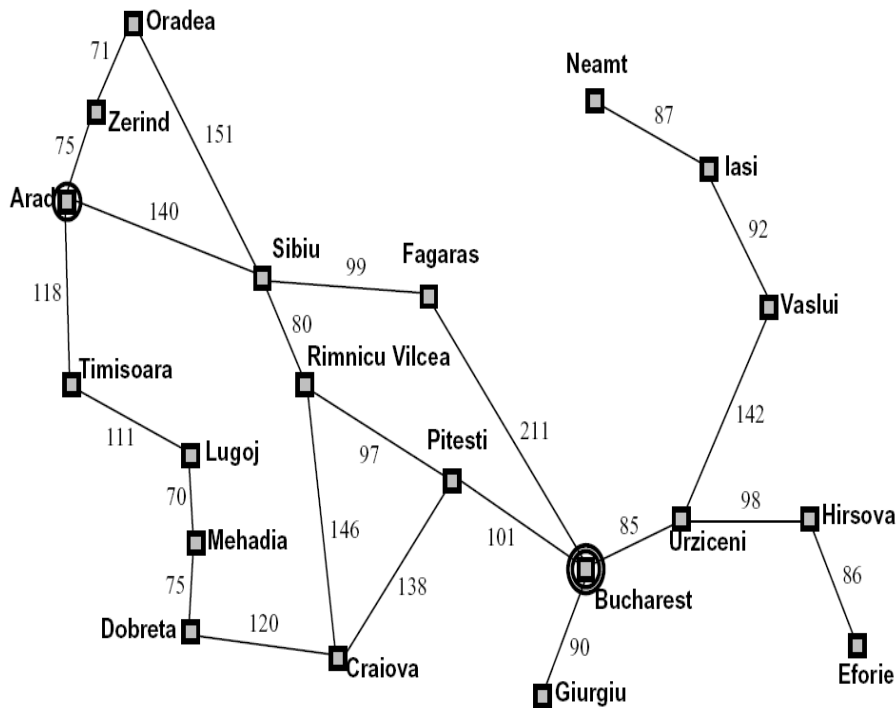
**310 Sibiu-Făgăraș-București (soluție DF)**

# Căutare uniformă după cost (uniform cost search – UCS)



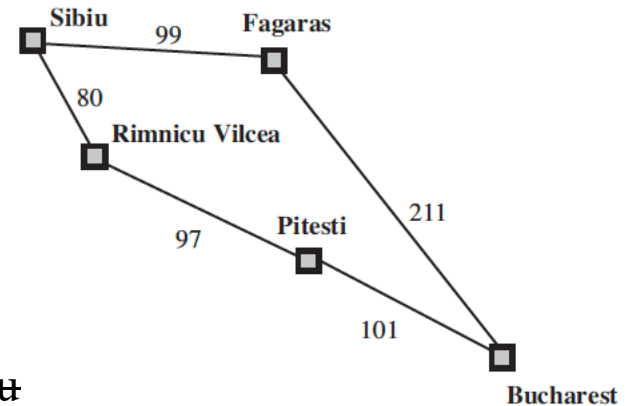
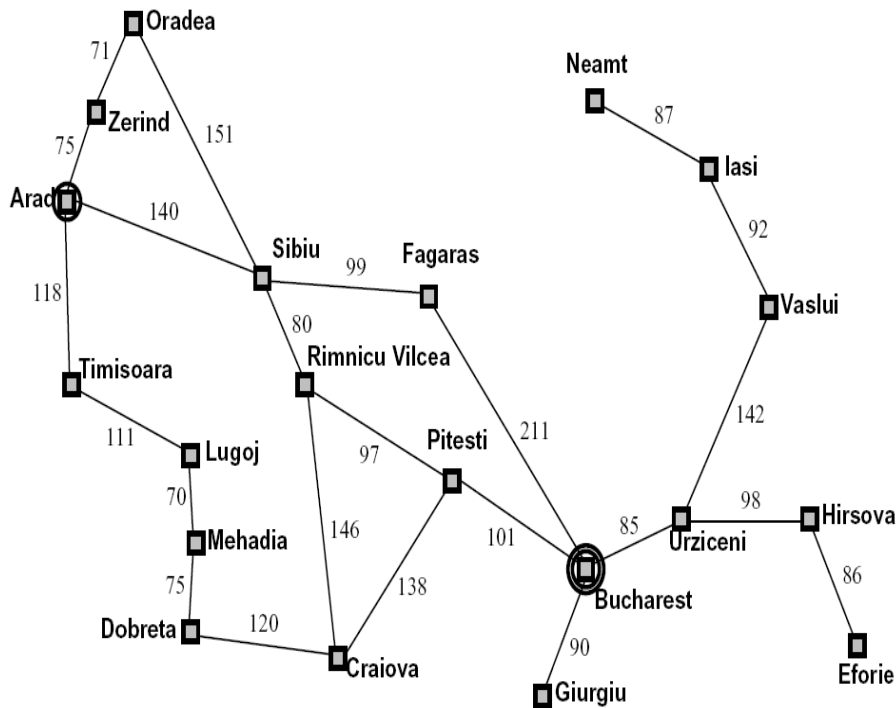
~~0 Sibiu~~  
~~80 Sibiu-RV~~  
~~99 Sibiu-Făgăraș~~  
~~160 Sibiu-RV-Sibiu~~  
~~177 Sibiu-RV-Pitești~~  
~~198 Sibiu-Făgăraș-Sibiu~~  
~~240 Sibiu-RV-Sibiu-RV~~  
~~259 Sibiu-RV-Sibiu-Făgăraș~~  
~~310 Sibiu-Făgăraș-București~~

# Căutare uniformă după cost (uniform cost search – UCS)



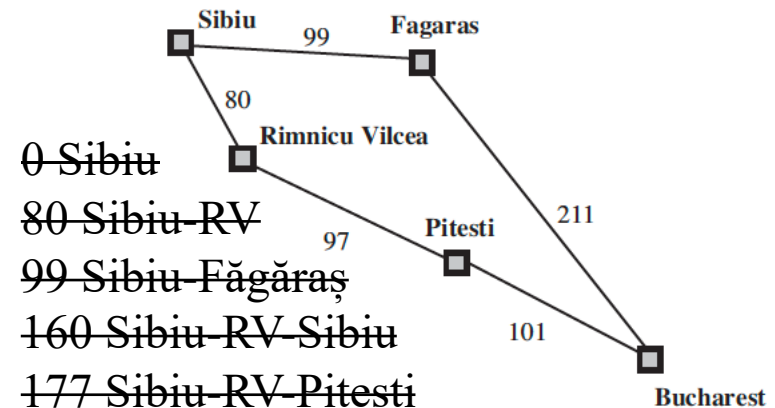
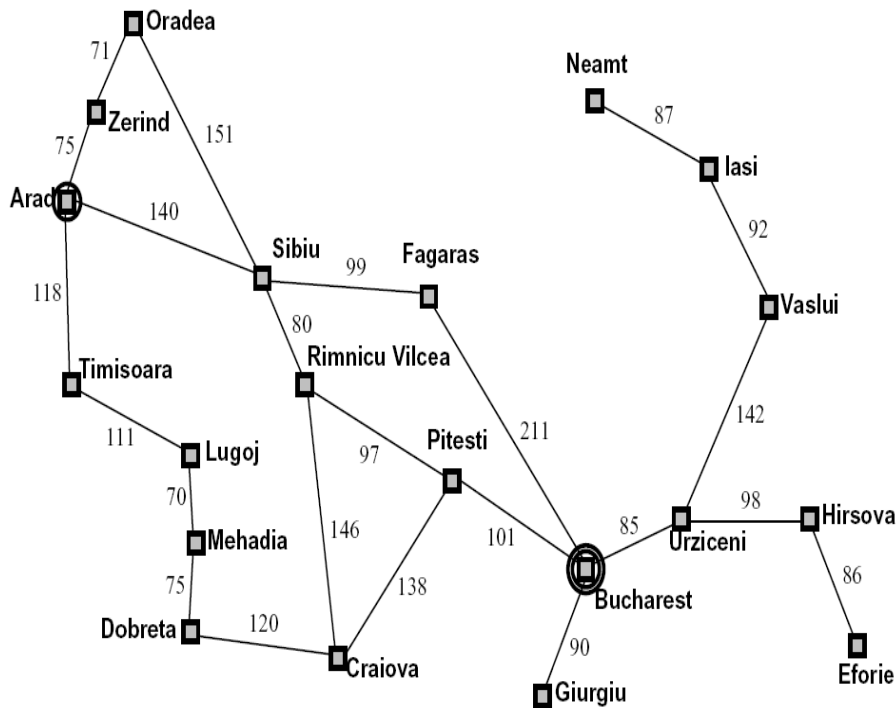
~~0 Sibiu~~  
~~80 Sibiu-RV~~  
~~99 Sibiu-Făgăraș~~  
~~160 Sibiu-RV-Sibiu~~  
~~177 Sibiu-RV-Pitești~~  
 198 Sibiu-Făgăraș-Sibiu  
 240 Sibiu-RV-Sibiu-RV  
 259 Sibiu-RV-Sibiu-Făgăraș  
 274 Sibiu-RV-Pitești-RV  
 278 Sibiu-RV-Pitești-București  
 310 Sibiu-Făgăraș-București

# Căutare uniformă după cost (uniform cost search – UCS)



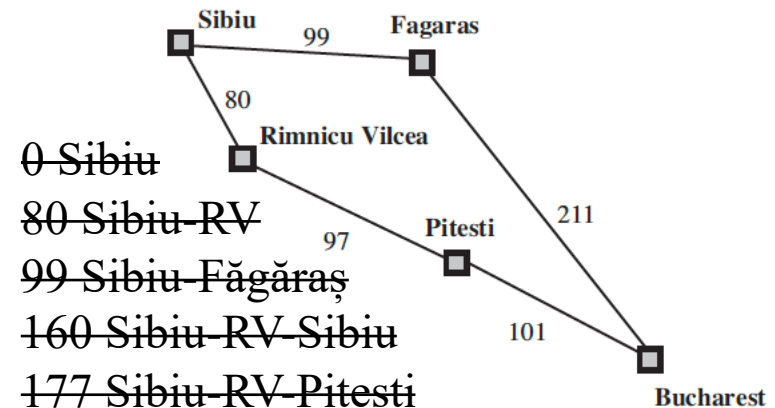
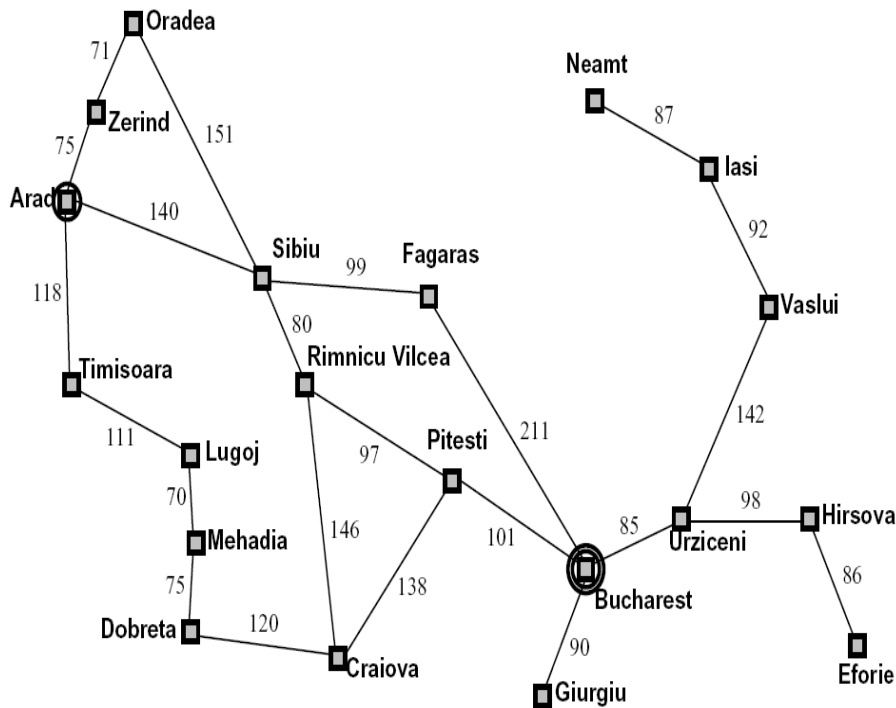
~~0 Sibiu~~  
~~80 Sibiu-RV~~  
~~99 Sibiu-Făgăraș~~  
~~160 Sibiu-RV-Sibiu~~  
~~177 Sibiu-RV-Pitești~~  
~~198 Sibiu-Făgăraș-Sibiu~~  
 240 Sibiu-RV-Sibiu-RV  
 259 Sibiu-RV-Sibiu-Făgăraș  
 274 Sibiu-RV-Pitești-RV  
 278 Sibiu-RV-Pitești-București  
 278 Sibiu-Făgăraș-Sibiu-RV  
 297 Sibiu-Făgăraș-Sibiu-Făgăraș  
 310 Sibiu-Făgăraș-București

# Căutare uniformă după cost (uniform cost search – UCS)



~~0 Sibiu~~  
~~80 Sibiu-RV~~  
~~99 Sibiu-Făgăraș~~  
~~160 Sibiu-RV-Sibiu~~  
~~177 Sibiu-RV-Pitești~~  
~~198 Sibiu-Făgăraș-Sibiu~~  
~~240 Sibiu-RV-Sibiu-RV~~  
~~259 Sibiu-RV-Sibiu-Făgăraș~~  
~~274 Sibiu-RV-Pitești-RV~~  
~~278 Sibiu-RV-Pitești-București~~  
~~278 Sibiu-Făgăraș-Sibiu-RV~~  
~~297 Sibiu-Făgăraș-Sibiu-Făgăraș~~  
~~310 Sibiu-Făgăraș-București~~  
~~320 Sibiu-RV-Sibiu-RV-Sibiu~~  
~~337 Sibiu-RV-Sibiu-RV-Pitești~~

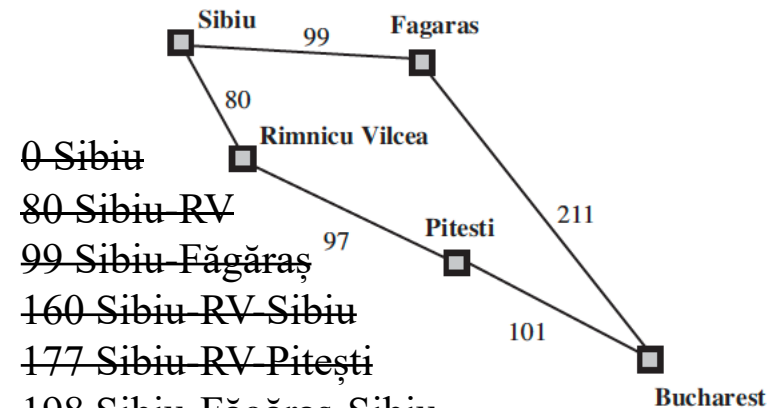
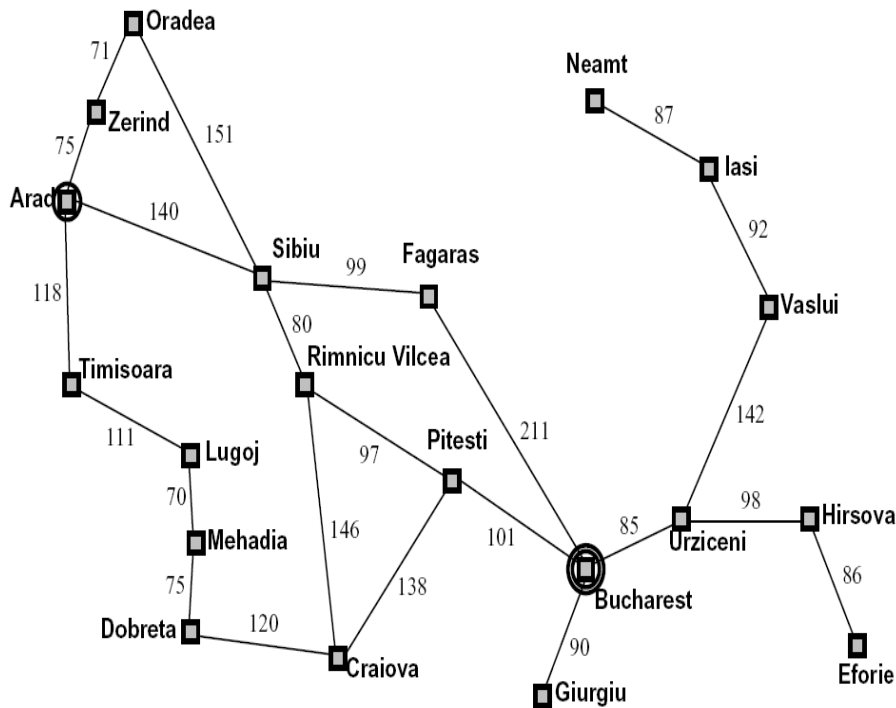
# Căutare uniformă după cost (uniform cost search – UCS)



~~0 Sibiu~~  
~~80 Sibiu-RV~~  
~~99 Sibiu-Făgăraș~~  
~~160 Sibiu-RV-Sibiu~~  
~~177 Sibiu-RV-Pitești~~  
~~198 Sibiu-Făgăraș-Sibiu~~  
~~240 Sibiu-RV-Sibiu-RV~~  
~~259 Sibiu-RV-Sibiu-Făgăraș~~  
~~274 Sibiu-RV-Pitești-RV~~  
~~278 Sibiu-RV-Pitești-București~~  
~~278 Sibiu-Făgăraș-Sibiu-RV~~  
~~297 Sibiu-Făgăraș-Sibiu-Făgăraș~~  
~~310 Sibiu-Făgăraș-București~~  
~~320 Sibiu-RV-Sibiu-RV-Sibiu~~  
~~337 Sibiu-RV-Sibiu-RV-Pitești~~  
~~358 Sibiu-RV-Sibiu-Făgăraș-Sibiu~~  
~~470 Sibiu-RV-Sibiu-Făgăraș-București~~

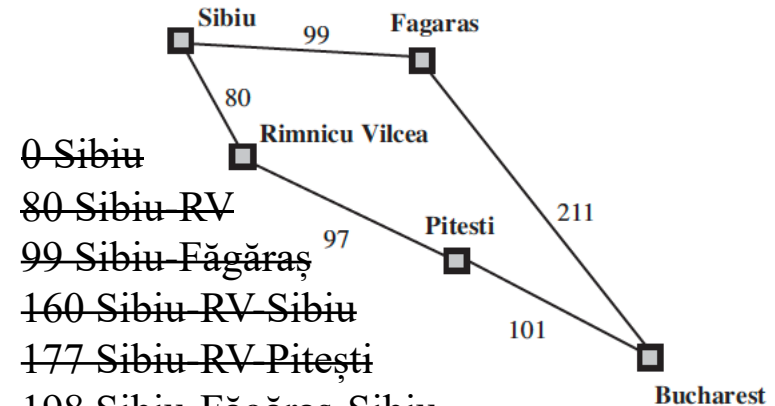
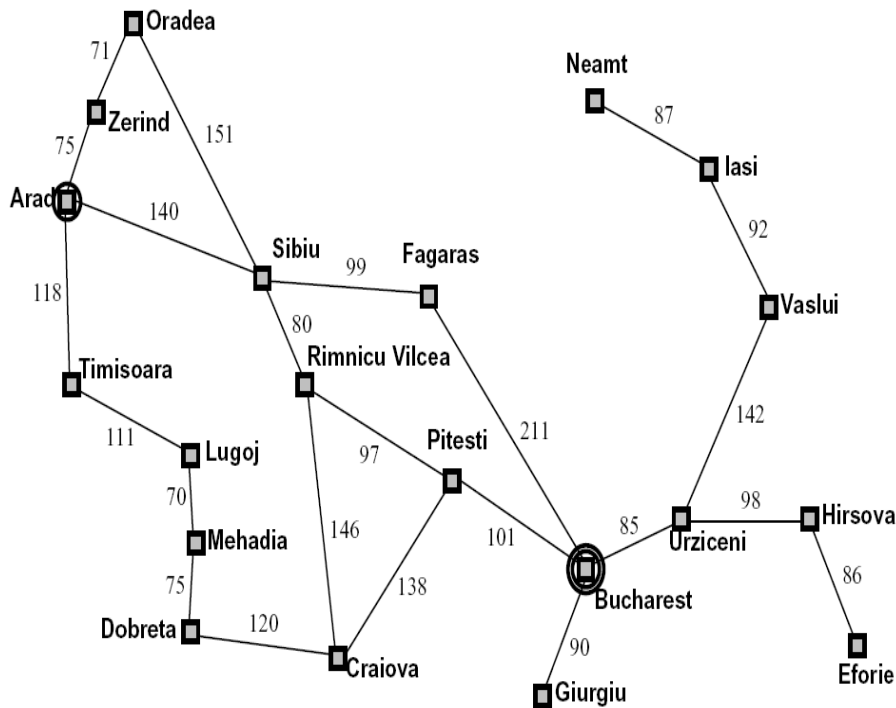


# Căutare uniformă după cost (uniform cost search – UCS)



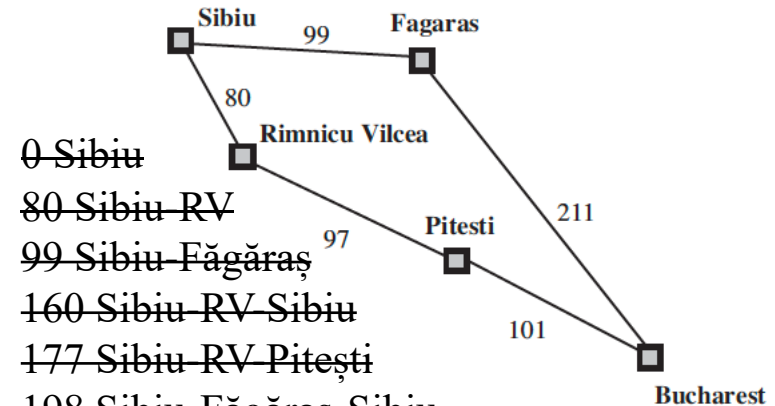
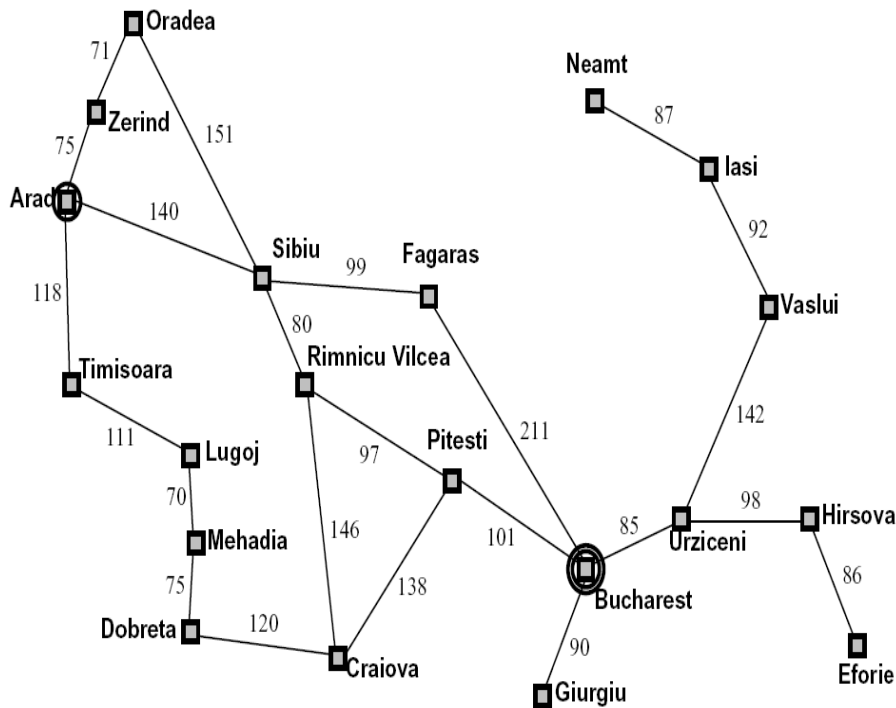
~~0 Sibiu~~  
~~80 Sibiu-RV~~  
~~99 Sibiu-Făgăraș~~  
~~160 Sibiu-RV-Sibiu~~  
~~177 Sibiu-RV-Pitești~~  
~~198 Sibiu-Făgăraș-Sibiu~~  
~~240 Sibiu-RV-Sibiu-RV~~  
~~259 Sibiu-RV-Sibiu-Făgăraș~~  
~~274 Sibiu-RV-Pitești-RV~~  
~~278 Sibiu-RV-Pitești-București~~  
~~278 Sibiu-Făgăraș-Sibiu-RV~~  
~~297 Sibiu-Făgăraș-Sibiu-Făgăraș~~  
~~310 Sibiu-Făgăraș-București~~  
~~320 Sibiu-RV-Sibiu-RV-Sibiu~~  
~~337 Sibiu-RV-Sibiu-RV-Pitești~~  
~~354 Sibiu-RV-Pitești-RV-Sibiu~~  
~~358 Sibiu-RV-Sibiu-Făgăraș-Sibiu~~  
~~371 Sibiu-RV-Pitești-RV-Pitești~~  
~~470 Sibiu-RV-Sibiu-Făgăraș-București~~

# Căutare uniformă după cost (uniform cost search – UCS)



~~0 Sibiu~~  
~~80 Sibiu-RV~~  
~~99 Sibiu-Făgăraș~~  
~~160 Sibiu-RV-Sibiu~~  
~~177 Sibiu-RV-Pitești~~  
~~198 Sibiu-Făgăraș-Sibiu~~  
~~240 Sibiu-RV-Sibiu-RV~~  
~~259 Sibiu-RV-Sibiu-Făgăraș~~  
~~274 Sibiu-RV-Pitești-RV~~  
**278 Sibiu-RV-Pitești-București (soluție UCS)**  
~~278 Sibiu-Făgăraș-Sibiu-RV~~  
~~297 Sibiu-Făgăraș-Sibiu-Făgăraș~~  
~~310 Sibiu-Făgăraș-București~~  
~~320 Sibiu-RV-Sibiu-RV-Sibiu~~  
~~337 Sibiu-RV-Sibiu-RV-Pitești~~  
~~354 Sibiu-RV-Pitești-RV-Sibiu~~  
~~358 Sibiu-RV-Sibiu-Făgăraș-Sibiu~~  
~~371 Sibiu-RV-Pitești-RV-Pitești~~  
~~470 Sibiu-RV-Sibiu-Făgăraș-București~~

# Căutare uniformă după cost (uniform cost search – UCS)



- ~~0 Sibiu~~
- ~~80 Sibiu-RV~~
- ~~99 Sibiu-Făgăraș~~
- ~~160 Sibiu-RV-Sibiu~~
- ~~177 Sibiu-RV-Pitești~~
- ~~198 Sibiu-Făgăraș-Sibiu~~
- ~~240 Sibiu-RV-Sibiu-RV~~
- ~~259 Sibiu-RV-Sibiu-Făgăraș~~
- ~~274 Sibiu-RV-Pitești-RV~~
- 278 Sibiu-RV-Pitești-București (soluție UCS)**
- ~~278 Sibiu-Făgăraș-Sibiu-RV~~
- ~~297 Sibiu-Făgăraș-Sibiu-Făgăraș~~
- ~~310 Sibiu-Făgăraș-București~~
- ~~320 Sibiu-RV-Sibiu-RV-Sibiu~~
- ~~337 Sibiu-RV-Sibiu-RV-Pitești~~
- ~~354 Sibiu-RV-Pitești-RV-Sibiu~~
- ~~358 Sibiu-RV-Sibiu-Făgăraș-Sibiu~~
- ~~371 Sibiu-RV-Pitești-RV-Pitești~~
- ~~470 Sibiu-RV-Sibiu-Făgăraș-București~~

**278 Sibiu-RV-Pitești-București (soluție UCS)**  
 UCS este, în esență, algoritmul lui Dijkstra aplicat până la primul nod scop.

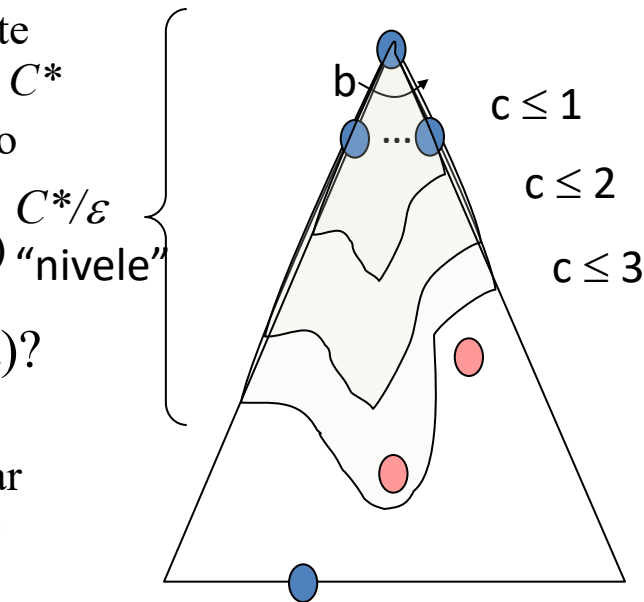
# UCS vs BF

1. Dacă toate costurile sunt egale atunci  $BF = UCS$
2. Când se testează dacă un nod conține o stare-scop?
  - BF testează când *se generează* un nou nod pentru explorare  
310 Sibiu-Făgăraș-București (soluție BF)
  - UCS testează când *se alege* un nou nod din frontieră pentru a fi explorat  
278 Sibiu-RV-Pitești-București

# Proprietăți ale căutării uniforme după cost

- Ce noduri expandează UCS?

- extinde nodul cu costul acumulat minim  $g(n)$
- înainte de a găsi soluția optimă, UCS va expanda toate nodurile cu cost mai mic decât costul soluției optime  $C^*$
- dacă fiecare arc costă cel puțin  $\varepsilon$ , atunci ajungem la o adâncime în jur de  $C^*/\varepsilon$
- complexitate timp  $O(b^{C^*/\varepsilon})$  (exponențial în adâncime)



- Complexitate spațiu (a memoriei pentru frontieră)?

- frontiera este o coadă de prioritate
- UCS păstrează în memorie toate nodurile generate dar neexpandate  $\Rightarrow$  aproximativ numărul de noduri de pe ultimul nivel, deci  $O(b^{C^*/\varepsilon})$

- Completitudine?

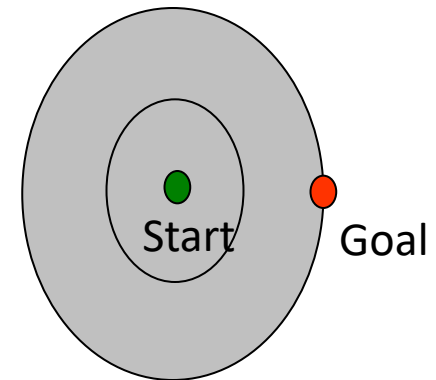
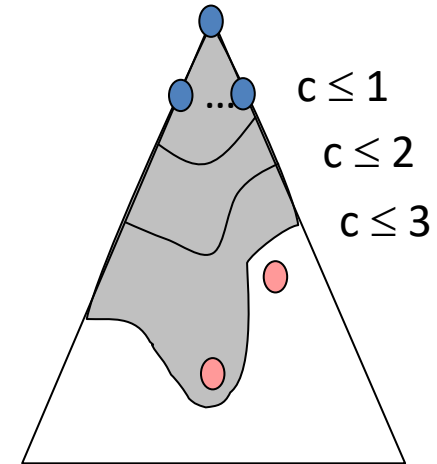
- DA (dacă soluția cea mai bună are cost finit iar costurile arcelor sunt pozitive)

- Optimalitate?

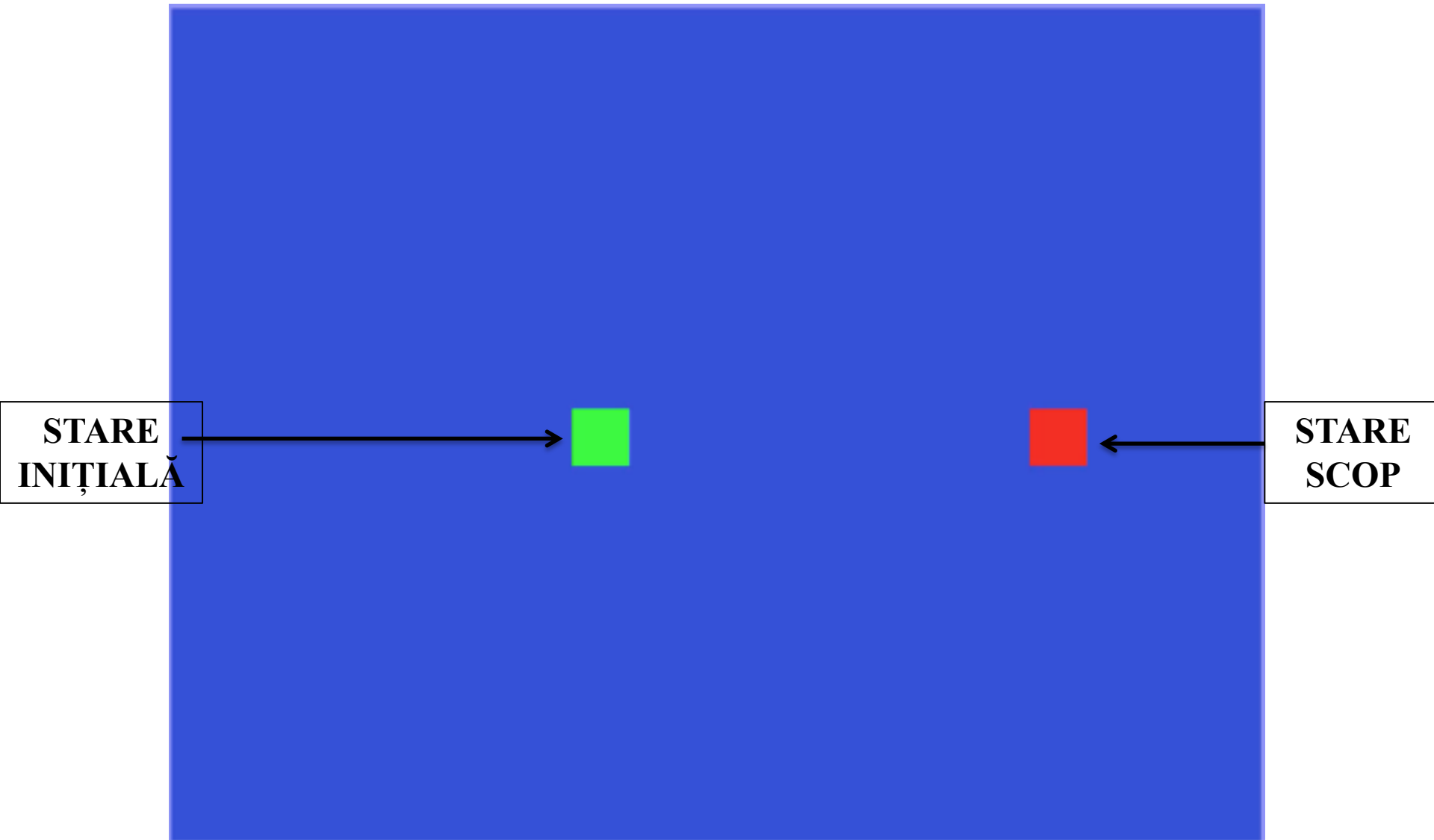
- DA! UCS returnează soluția cu cost minim, nodurile sunt expandate în ordine crescătoare a costului

# Avantaje/dezavantaje ale căutării uniforme după cost

- UCS explorează drumuri în ordinea costurilor
- Avantaje: UCS este complet și optimal!
- Dezavantaje:
  - explorează drumurile în toate direcțiile (căutare uniformă)
  - nu are nicio informație despre poziția nodului scop



# BF vs DF vs UCS

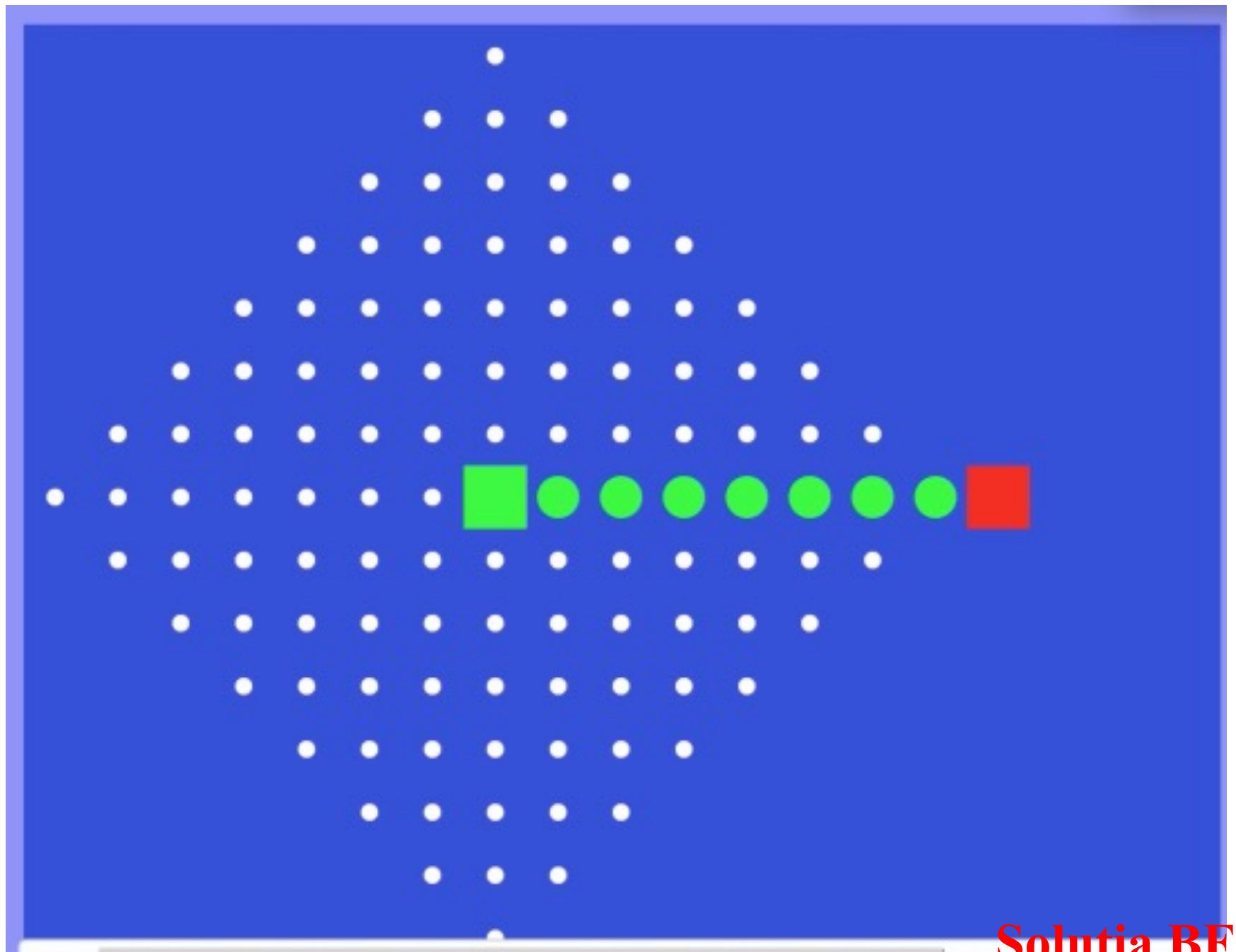


# Soluția BF



**Soluția BF**



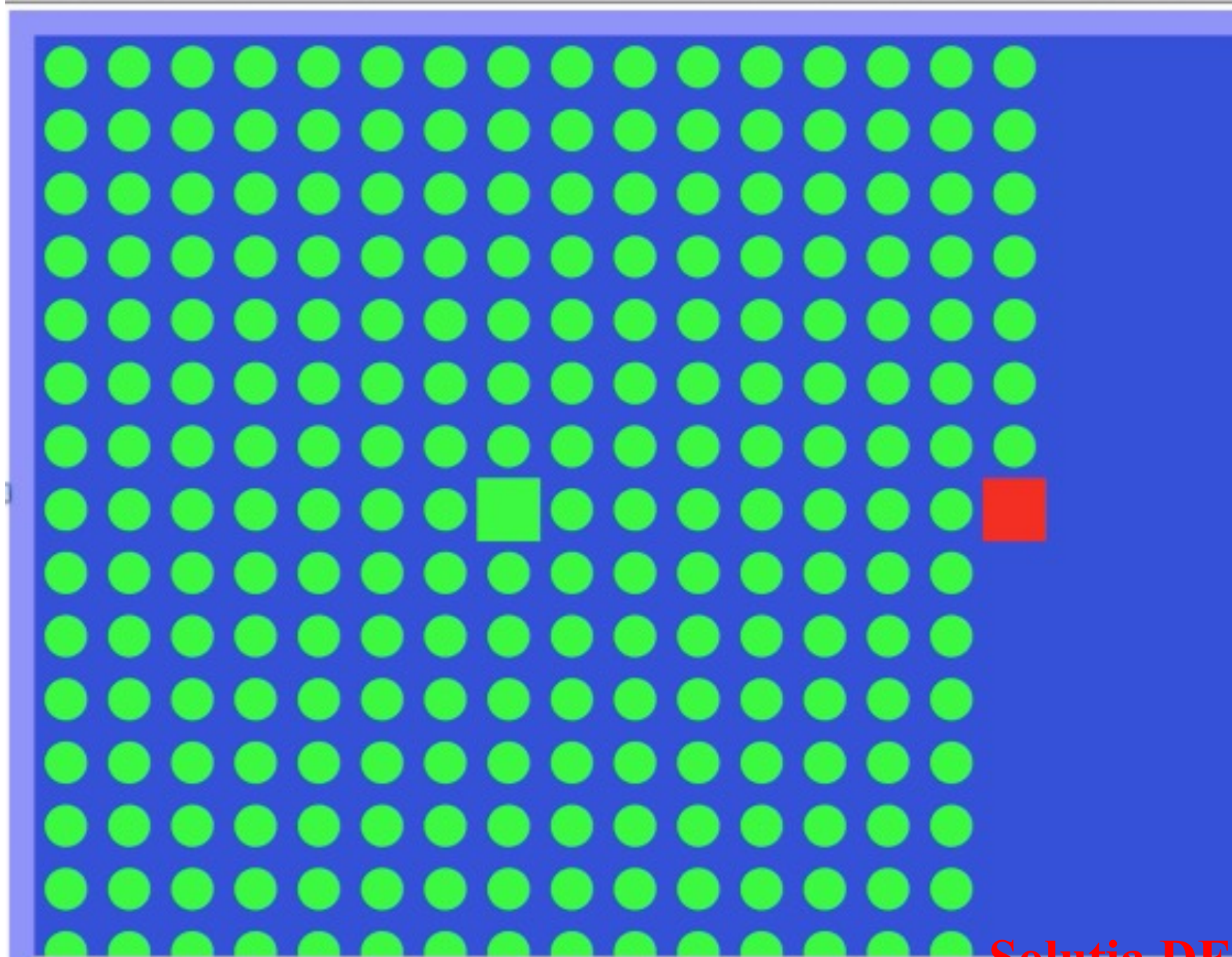


**Soluția BF**

# Soluția DF



**Soluția DF**

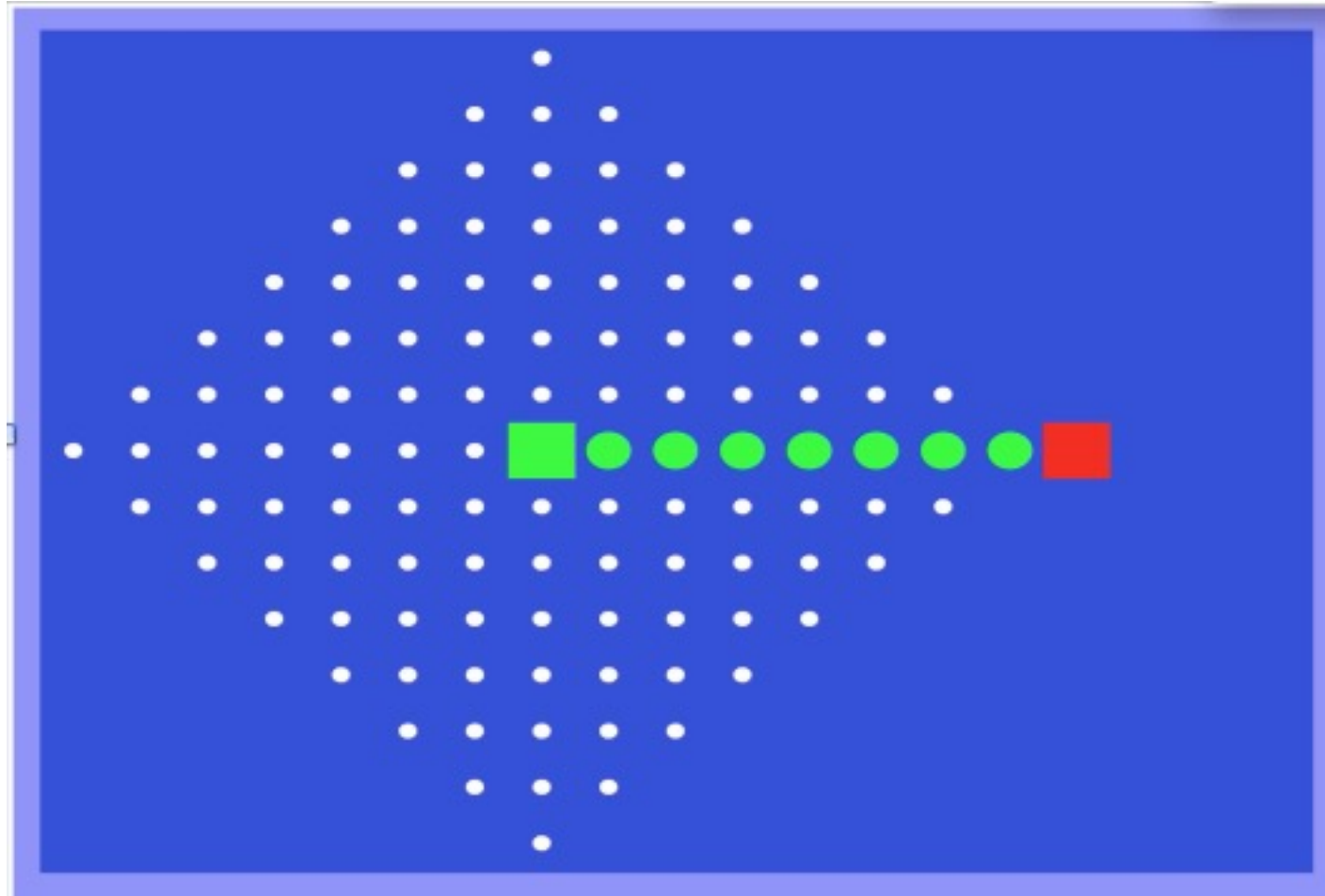


**Soluția DF**

# Soluția UCS

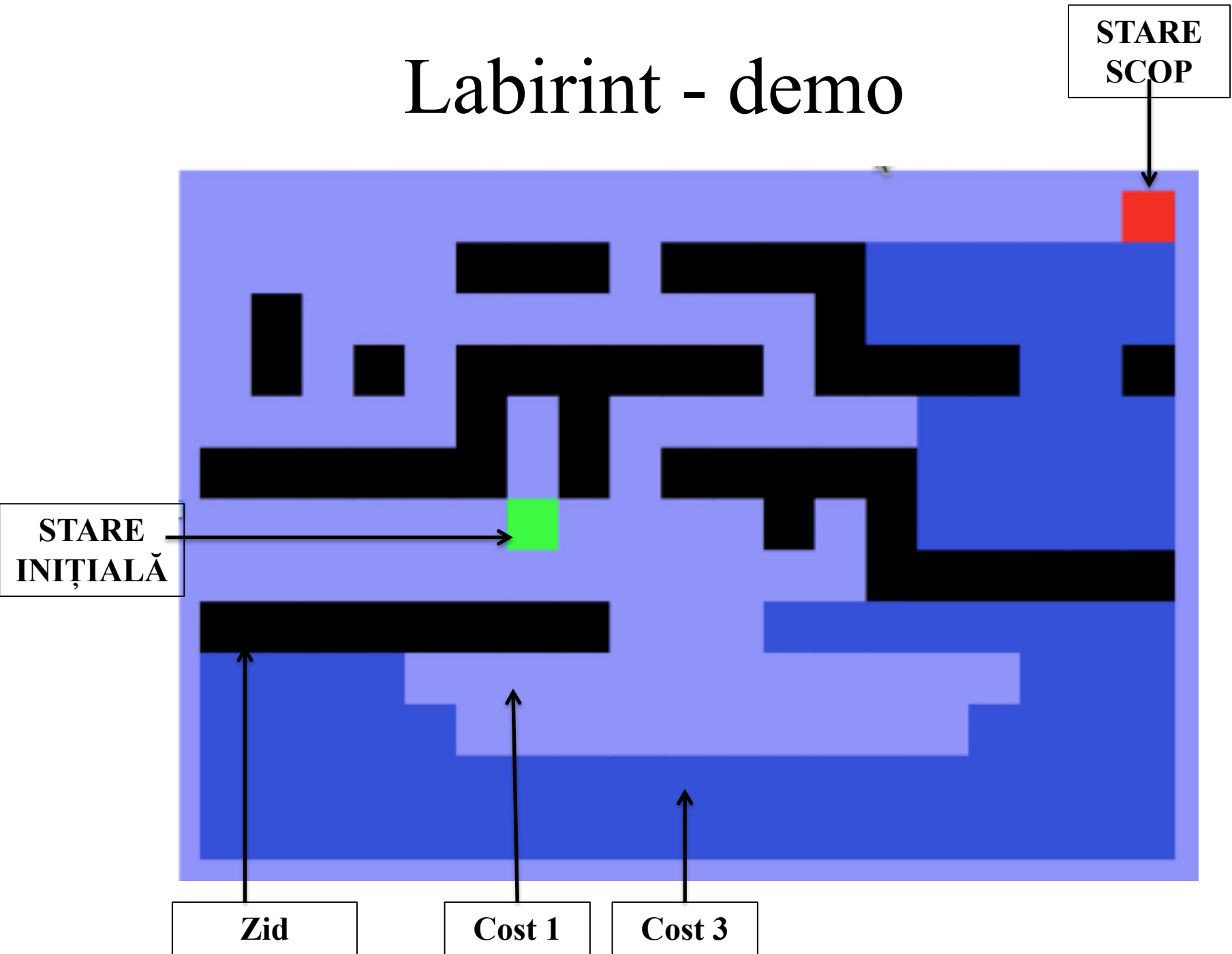


# Soluția UCS



**Soluția UCS**

# Labirint - demo



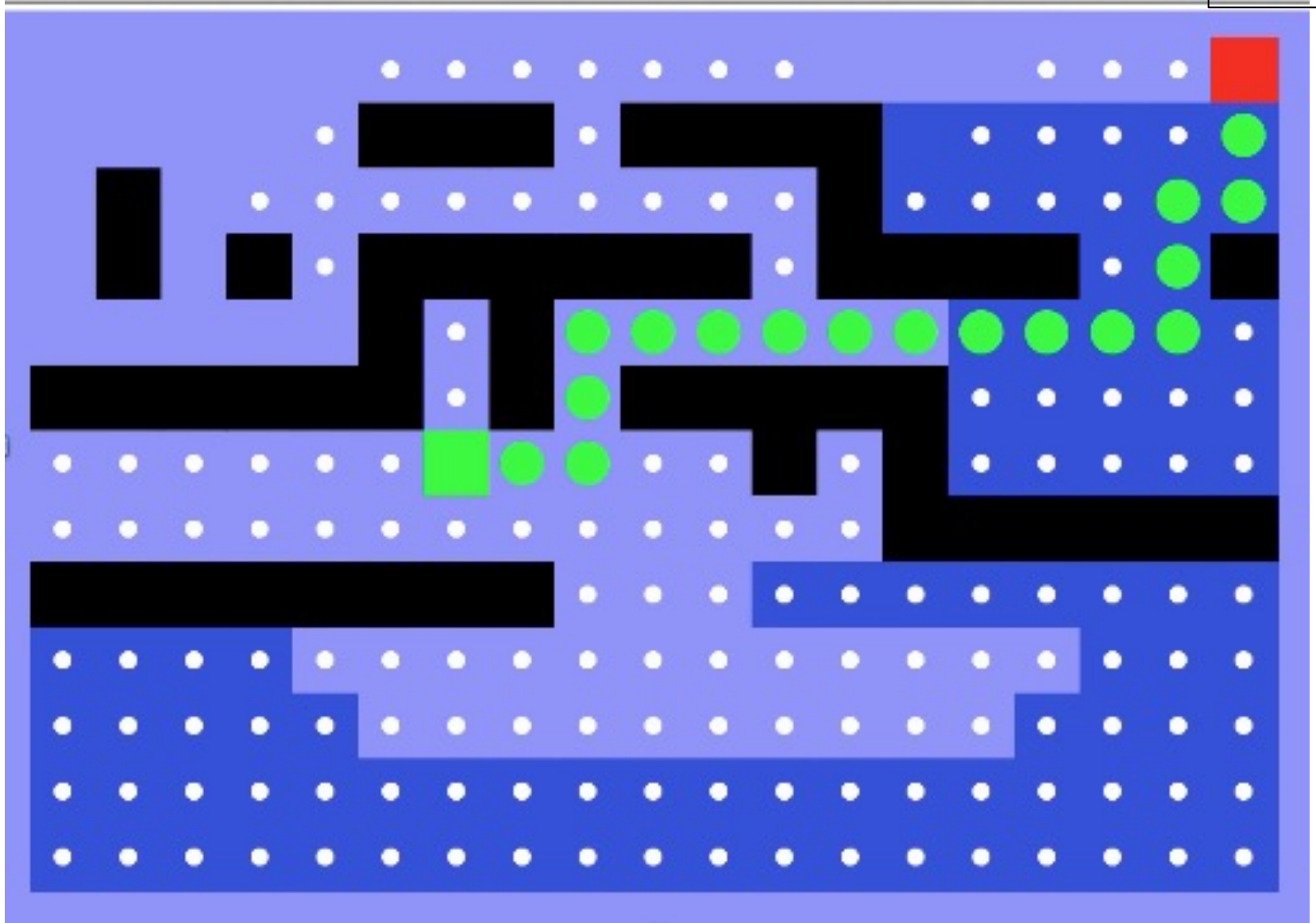
# Labirint – demo Breadth First search





# Labirint – demo Breadth First search

Cost 36



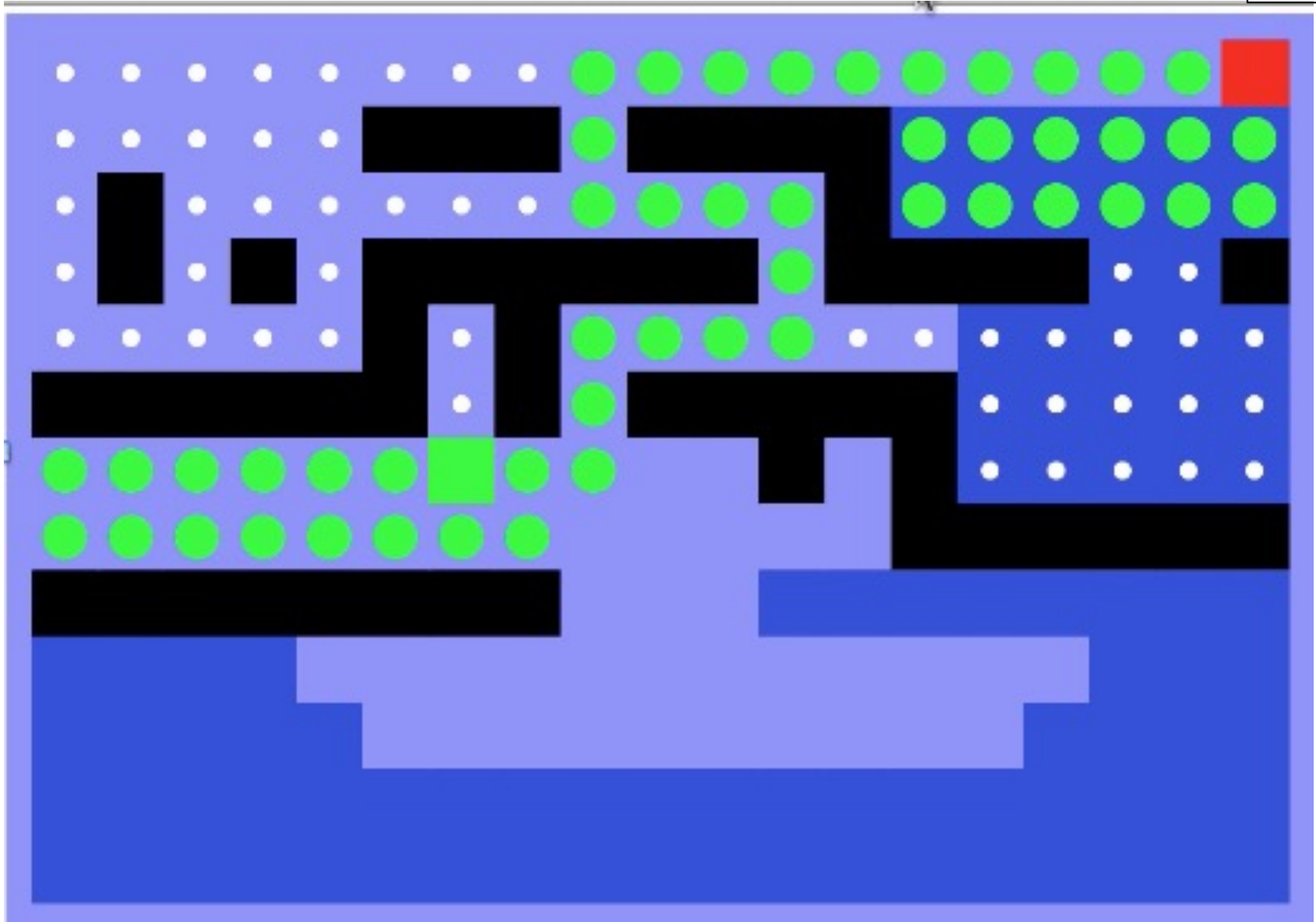


# Labirint – demo Depth First search



# Labirint – demo Depth First search

Cost 74

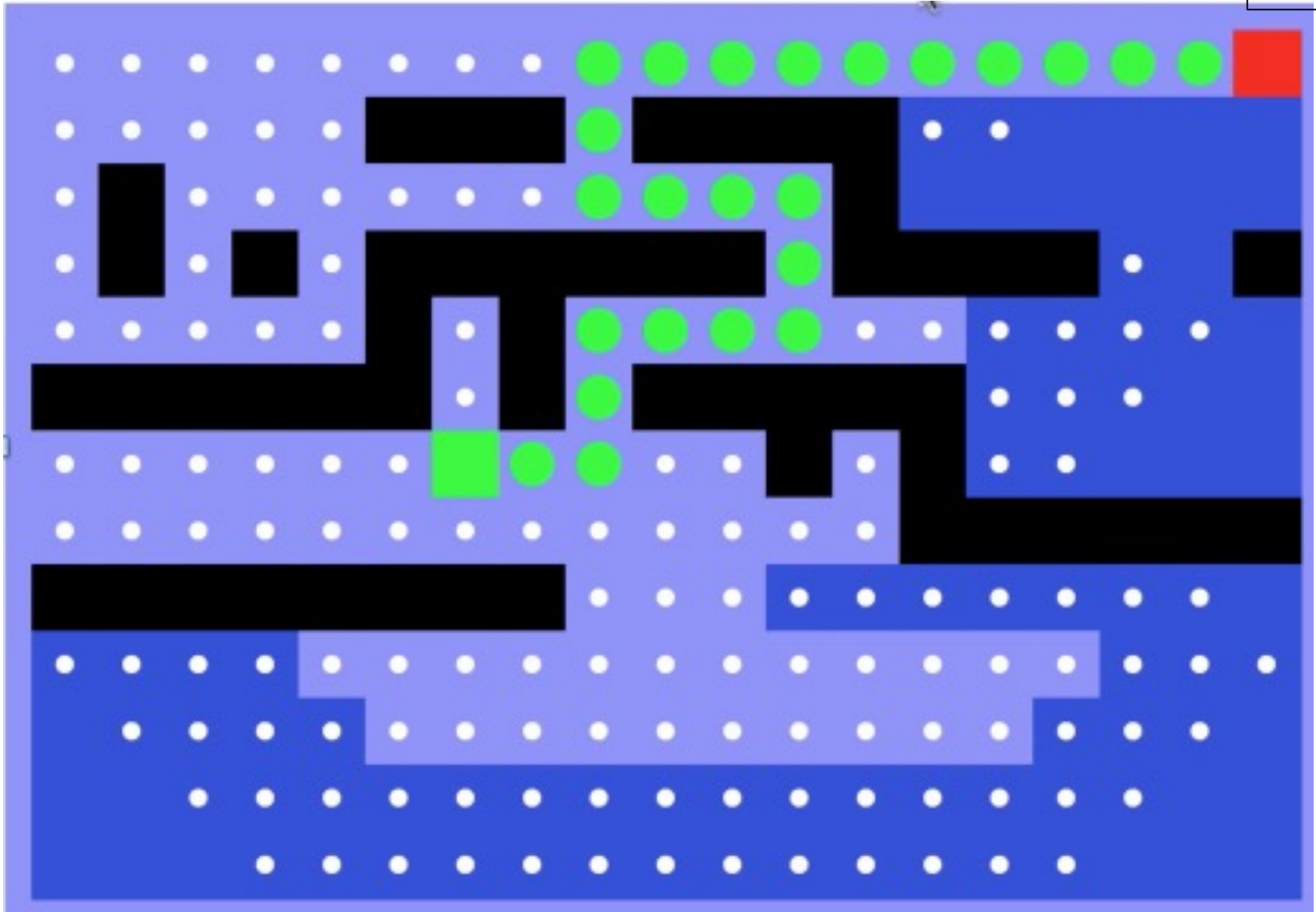


# Labirint – demo Uniform Cost Search

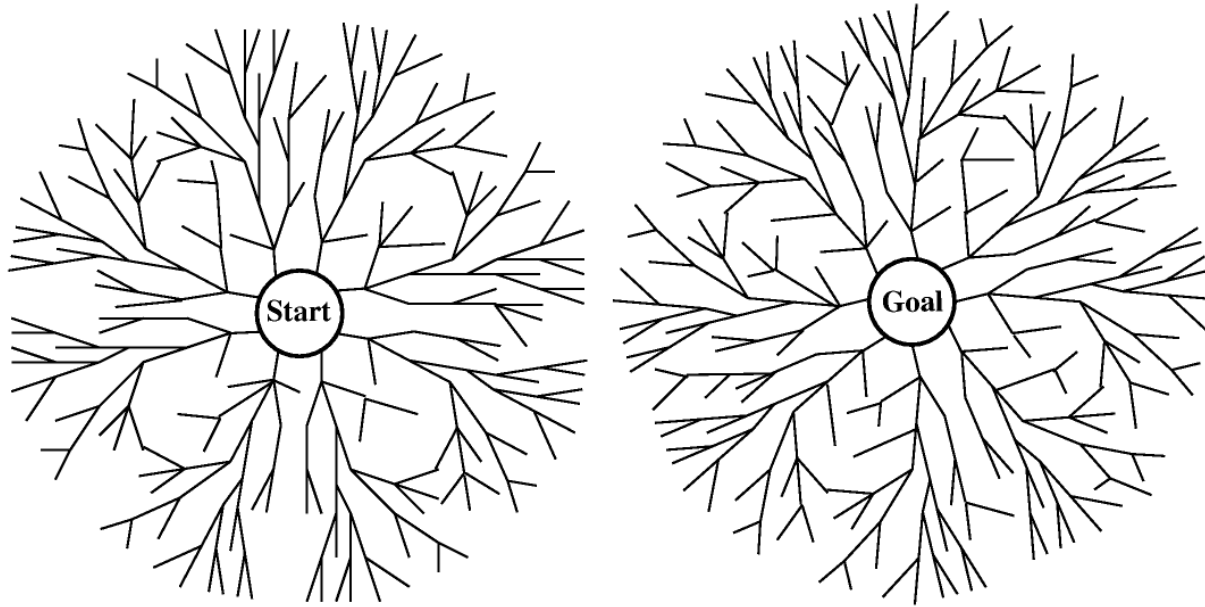


# Labirint – demo Uniform Cost Search

Cost 24

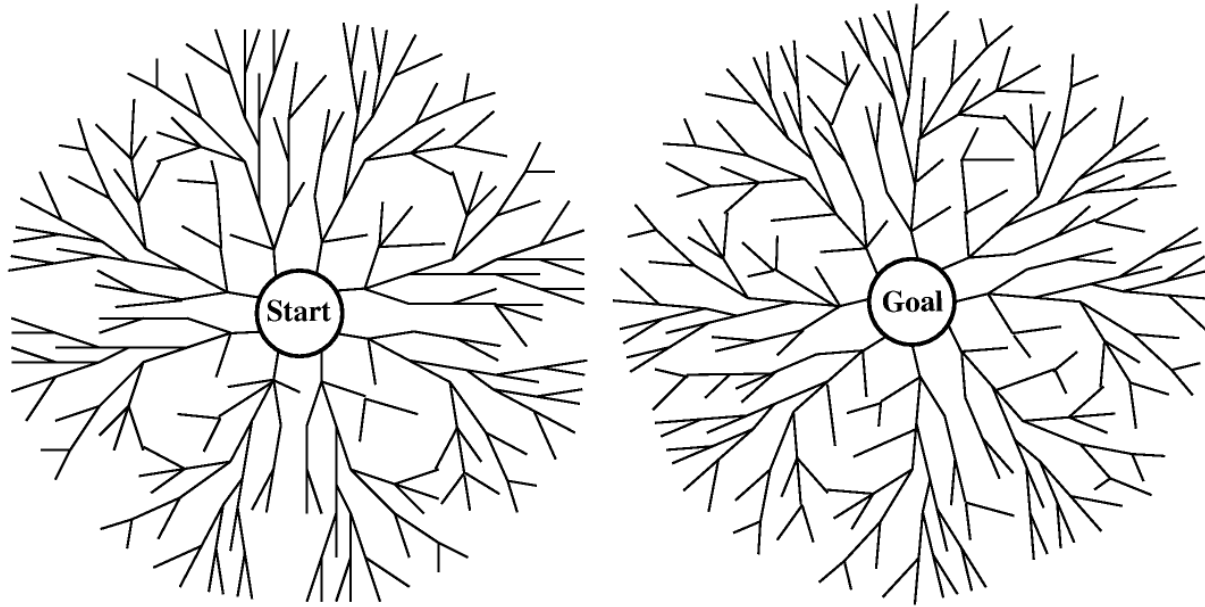


# Căutare bidirecțională



Realizează căutarea în paralel, pornind din starea inițială și starea finală.

# Căutare bidirecțională

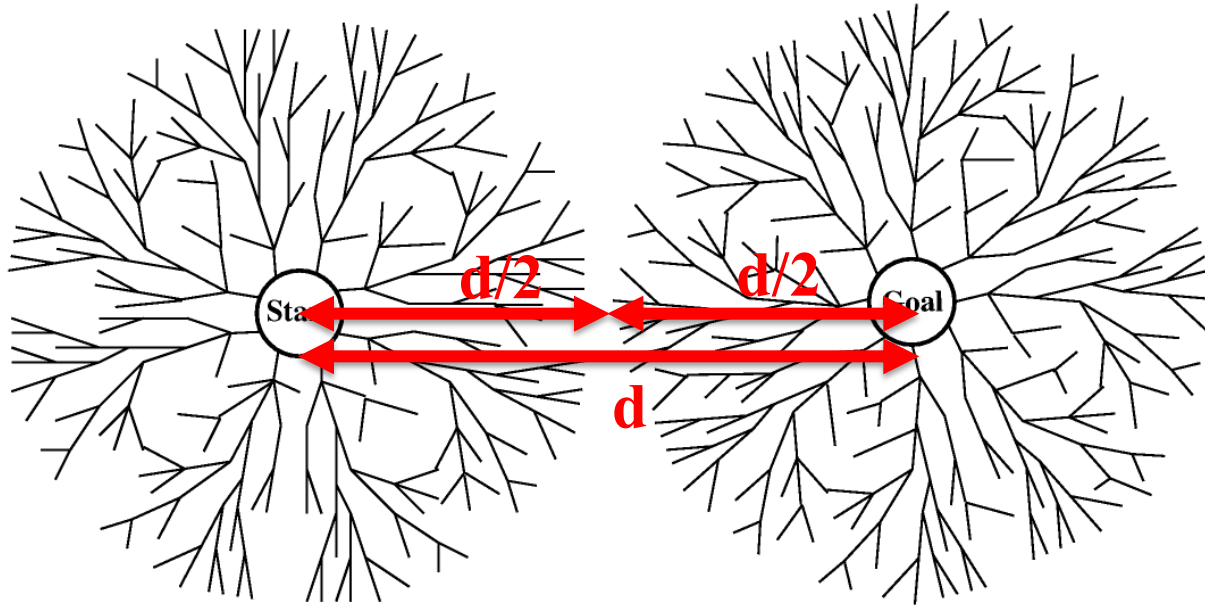


$$O(b^{d/2}) + O(b^{d/2}) = 2O(b^{d/2}) = O(b^{d/2})$$

Realizează căutarea în paralel, pornind din starea inițială și starea finală.

Dacă soluția se găsește la nivelul  $d$  iar factorul de ramificare este  $b$  în ambele căutări, putem explora numai  $O(b^{d/2})$  noduri în loc de  $O(b^d)$ .

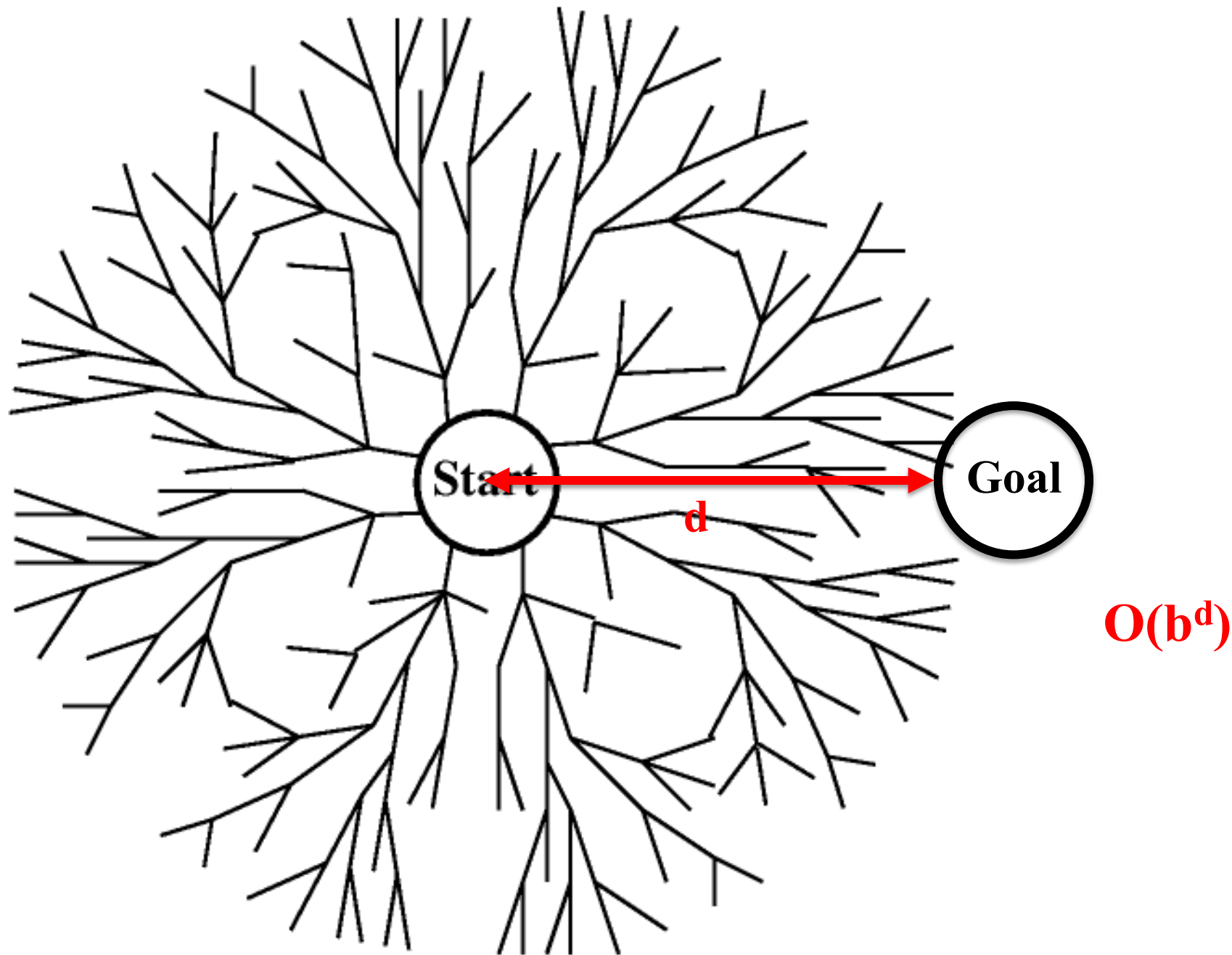
# Căutare bidirecțională



$$O(b^{d/2}) + O(b^{d/2}) = 2O(b^{d/2}) = O(b^{d/2})$$

Realizează căutarea în paralel, pornind din starea inițială și starea finală.

Dacă soluția se găsește la nivelul  $d$  iar factorul de ramificare este  $b$  în ambele căutări, putem explora numai  $O(b^{d/2})$  noduri în loc de  $O(b^d)$ .

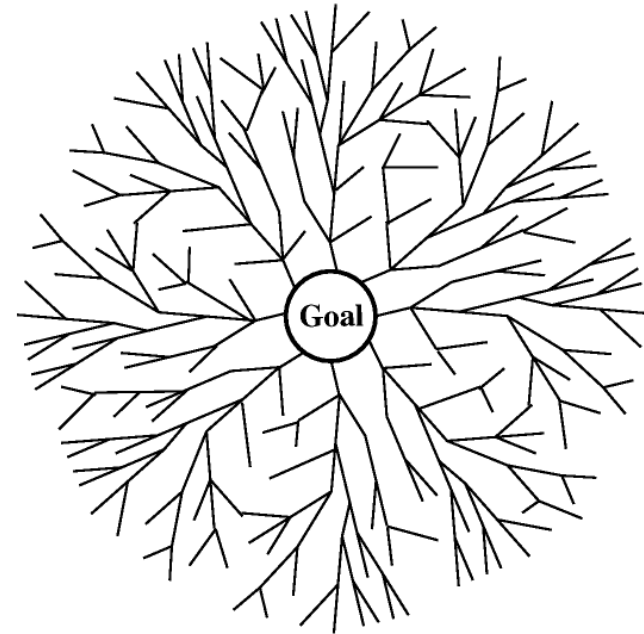




# Căutare bidirecțională

## Căutarea din starea finală:

- pentru a porni căutarea inversă trebuie să putem genera predecesorii unei stări.
- acest lucru este posibil dacă acțiunile sunt reversibile
- în aceste cazuri putem construi graful de căutare invers
- pentru probleme de căutare cu acțiuni reversibile este ușor de găsit (8-puzzle, misionari și canibali)



## Limitări

- nu întotdeauna putem genera ușor stările predecesoare
- uneori starea finală nu este cunoscută explicit

# Compararea strategiilor de căutare neinformată

| Criteriu       | Căutare în lăţime (BF) | Căutare uniformă pe bază de cost (UCS) | Căutare în adâncime (DF) | Căutare în adâncime limitată (DL) | Căutare iterativă în adâncime (ID) | Căutare bidirecţională |
|----------------|------------------------|----------------------------------------|--------------------------|-----------------------------------|------------------------------------|------------------------|
| Completitudine | DA <sup>a</sup>        | DA <sup>a,b</sup>                      | NU                       | NU                                | DA <sup>a</sup>                    | DA <sup>a</sup>        |
| Timp           | $O(b^d)$               | $O(b^{C^*/\varepsilon})$               | $O(b^m)$                 | $O(b^l)$                          | $O(b^d)$                           | $O(b^{d/2})$           |
| Spaţiu         | $O(b^d)$               | $O(b^{C^*/\varepsilon})$               | $O(b^m)$                 | $O(b^l)$                          | $O(b^d)$                           | $O(b^{d/2})$           |
| Optimalitate   | DA <sup>c</sup>        | DA                                     | NU                       | NU                                | DA <sup>c</sup>                    | DA <sup>c</sup>        |

b – branching factor (factor de ramificare)

d – adâncimea minimă a unei soluţii

m – adâncimea maximă în arborele de căutare

DA<sup>a</sup> – complet dacă b este finit

DA<sup>b</sup> – complet dacă fiecare arc are costul  $\geq \varepsilon$

DA<sup>c</sup> – optimal dacă toate costurile sunt egale = 1

# Compararea strategiilor de căutare neinformată

| Criteriu       | Căutare în lăţime (BF) | Căutare uniformă pe bază de cost (UCS) | Căutare în adâncime (DF) | Căutare în adâncime limitată (DL) | Căutare iterativă în adâncime (ID) | Căutare bidirecţională |
|----------------|------------------------|----------------------------------------|--------------------------|-----------------------------------|------------------------------------|------------------------|
| Completitudine | DA <sup>a</sup>        | DA <sup>a,b</sup>                      | NU                       | NU                                | DA <sup>a</sup>                    | DA <sup>a</sup>        |
| Timp           | $O(b^d)$               | $O(b^{C^*/\epsilon})$                  | $O(b^m)$                 | $O(b^l)$                          | $O(b^d)$                           | $O(b^{d/2})$           |
| Spaţiu         | $O(b^d)$               | $O(b^{C^*/\epsilon})$                  | $O(bm)$                  | $O(bl)$                           | $O(bd)$                            | $O(b^{d/2})$           |
| Optimalitate   | DA <sup>c</sup>        | DA                                     | NU                       | NU                                | DA <sup>c</sup>                    | DA <sup>c</sup>        |

b – branching factor (factor de ramificare)

d – adâncimea minimă a unei soluţii

m – adâncimea maximă în arborele de căutare

DA<sup>a</sup> – complet dacă b este finit

DA<sup>b</sup> – complet dacă fiecare arc are costul  $\geq \epsilon$

DA<sup>c</sup> – optimal dacă toate costurile sunt egale = 1

# Evitarea stărilor repetate

- idee esențială în algoritmi de căutare
- importantă când acțiunile sunt reversibile
- evită bucle infinite
- putem memora stările deja explorate și nu mai expandăm noduri care conțin aceste stări
- poate reduce exponențial numărul de noduri generate

# Open List și Closed List

## Open List (Frontiera)

- conține **nodurile generate** dar încă neexpandate
- se mai numește frontieră sau fringe
- implementarea depinde de strategia de căutare:
  - coadă (FIFO) → Breadth-First Search
  - stivă (LIFO) → Depth-First Search
  - coadă de priorități → Uniform Cost Search / A\*

## Closed List

- memorează **stările explorate**
- evită explorarea repetată a stărilor
- înainte de a expanda un nod verificăm dacă starea se află deja în Closed List

# Algoritmul de căutare în arbore (TREE-SEARCH)

funcția **TREE-SEARCH** (**problema**, **strategie**) **returnează** o soluție sau eșec

inițializează **frontiera (Open list)** folosind **starea inițială a problemei**

**ciclează**

**dacă frontiera** este vidă **atunci returnează** eșec

alege un **nod frunză** conform **strategiei** și elimină-l din **frontieră**

**dacă nodul** conține o **stare scop**

**atunci returnează** soluția corespunzătoare

expandează nodul ales, adăugând nodurile succesori generate în **frontieră**

# Algoritmul de căutare în graf (GRAPH-SEARCH)

**funcția GRAPH-SEARCH** (**problema**, **strategie**) **returnează** o soluție sau eșec

inițializează **frontiera (Open list)** folosind **starea inițială a problemei**

inițializează mulțimea (Closed List) de stări explorate (vizitate) cu mulțimea vidă  
**ciclează**

**dacă frontiera** este vidă **atunci returnează** eșec

alege un **nod frunză** conform **strategiei** și elimină-l din **frontieră**

**dacă nodul** conține o **stare scop**

**atunci returnează** soluția corespunzătoare

**dacă starea nodului curent este în Closed List**

**atunci continuă**

**adaugă starea nodului curent la Closed List**

**expandează nodul ales, adăugând nodurile succesori generate în frontieră**

# Căutare în arbore (TREE-SEARCH) vs căutare în graf (GRAPH-SEARCH)

## Tree Search

- nu memorează stările vizitate
- aceeași stare poate fi generată și expandată (prin intermediul nodurilor) de mai multe ori
- poate produce multe explorări redundante

## Graph Search

- memorează stările deja explorate (Closed List)
- evită reexplorarea aceleiași stări
- fiecare stare este expandată cel mult o dată



# Complexitatea căutării în graf (Graph Search)

## Complexitate timp $O(\text{numărul de stări})$

- fiecare stare este explorată cel mult o dată
- în cel mai rău caz sunt explorate toate stările spațiului de căutare
- în practică, adesea mult mai mic decât  $O(b^d)$
- aceeași limită superioară indiferent de strategia de căutare

## Complexitate spațiu $O(\text{numărul de stări})$

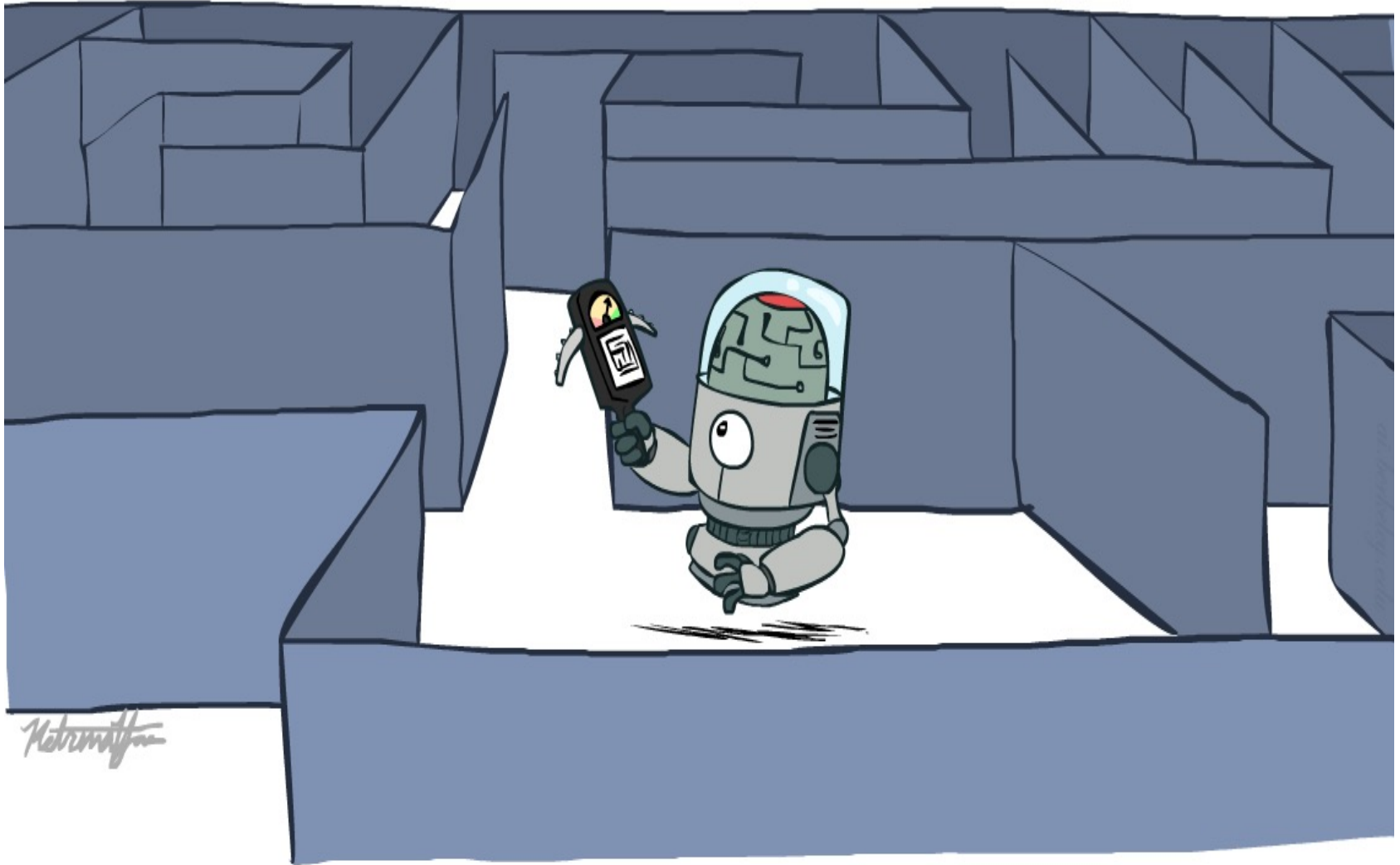
- trebuie memorate toate stările explorate
- spațiul  $\approx$  numărul de noduri generate = *numărul de stări*
- poate deveni foarte mare în practică
- reduce reexplorarea stărilor, dar necesită multă memorie pentru a păstra lista Closed.

# Optimalitatea căutării în graf (Graph Search)

## **Graph Search nu este întotdeauna optimal**

- atunci când este detectată o stare repetată, algoritmul a găsit un alt drum către aceeași stare
- în varianta standard de Graph Search, se păstrează primul drum găsit, iar celelalte sunt ignorate
- însă noul drum ar putea avea cost mai mic
- în acest caz algoritmul poate pierde soluția optimă
- este optimal doar dacă strategia de căutare garantează că prima soluție găsită este optimă (BF – dacă toate costurile sunt egale, UCS – dacă costurile sunt pozitive)

# Căutare informată



# Căutare informată (euristică)

- o strategie de căutare informată folosește cunoștințe specifice problemei, pe lângă descrierea formală a problemei
- utilizează euristici (estimări / „ghiciri inteligente”) pentru a ghida căutarea către starea scop
- scopul este reducerea numărului de noduri explorate
- euristica estimează cât de aproape este o stare de scop

# Căutare informată vs căutare neinformată

alege un **nod frunză** conform **strategiei** și elimină-l din **frontieră**

**Strategia de căutare** = precizează ordinea în care expandăm/explorăm nodurile

- funcție de evaluare  $f(n)$  – evaluează nodul  $n$
- alege un nod  $n$  pe baza lui  $f(n)$

## Căutare neinformată

- $f(n)$  nu codează informații specifice despre problemă
- BF:  $f(n)$  = primul nod dintr-o coadă, nod de adâncime minimă
- DF:  $f(n)$  = primul nod dintr-o stivă, nod de adâncime maximă
- UCS:  $f(n)$  = costul nodului, coadă de priorități

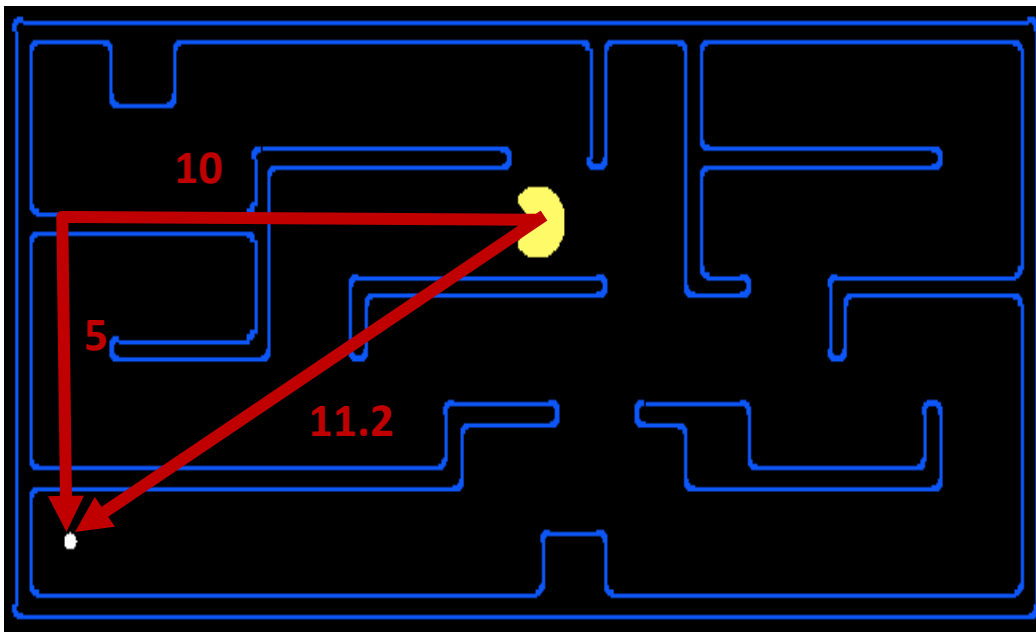
## Căutare informată

- $f(n)$  codează informații specifice despre problemă
- includem în  $f(n)$  și euristici (nu le avem la căutare neinformată)

# Funcții euristice

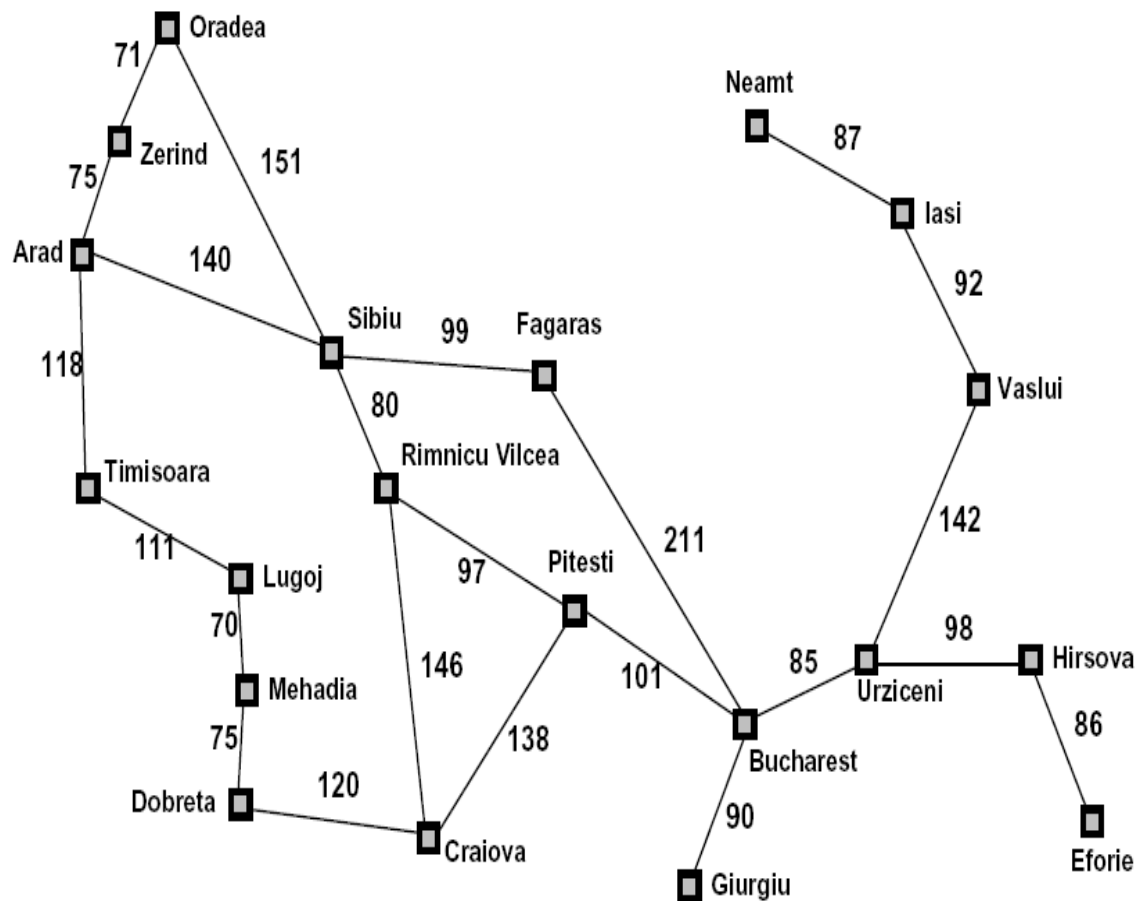
O euristică este:

- o funcție  $h$  care *estimează* cât de aproape suntem de o stare-scop;
- $h(n) \geq 0$ , pentru orice stare  $n$ ;
- **$h(\text{stare-scop}) = 0$ ;**
- specifică pentru fiecare problemă;
- exemple: distanța Manhattan, distanța Euclideană pentru găsirea celui mai scurt drum;



# Exemplu de funcție euristică

$h(x)$  = distanța în linie dreaptă între orașul  $x$  și București



| Straight-line distance to Bucharest |     |
|-------------------------------------|-----|
| Arad                                | 366 |
| Bucharest                           | 0   |
| Craiova                             | 160 |
| Dobreta                             | 242 |
| Eforie                              | 161 |
| Fagaras                             | 178 |
| Giurgiu                             | 77  |
| Hirsova                             | 151 |
| Iasi                                | 226 |
| Lugoj                               | 244 |
| Mehadia                             | 241 |
| Neamt                               | 234 |
| Oradea                              | 380 |
| Pitesti                             | 98  |
| Rimnicu Vilcea                      | 193 |
| Sibiu                               | 253 |
| Timisoara                           | 329 |
| Urziceni                            | 80  |
| Vaslui                              | 199 |
| Zerind                              | 374 |

$h(x)$

# Exemplu de funcție euristică

- Pentru problema 8-puzzle putem folosi câteva euristici:
  - $h_1$ : numărul de piese (galbene) așezate greșit față de starea-scop
  - $h_2$ : suma distanțelor pieselor față de poziția lor în starea scop (suma distanțelor Manhattan)

|   |   |   |
|---|---|---|
| 2 |   | 3 |
| 1 | 8 | 4 |
| 7 | 6 | 5 |

Stare inițială

$$h_1 = 3$$

$$h_2 = 1 + 1 + 0 + 0 + 0 + 0 + 0 + 1 = 3$$

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 |   | 4 |
| 7 | 6 | 5 |

Stare scop



# Exemplu de funcție euristică

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 |   | 4 |
| 7 | 6 | 5 |

Stare scop

$h_1 = 3$

$h_2 = 3$

|   |   |   |
|---|---|---|
| 2 |   | 3 |
| 1 | 8 | 4 |
| 7 | 6 | 5 |

STÂNGA

|   |   |   |
|---|---|---|
|   | 2 | 3 |
| 1 | 8 | 4 |
| 7 | 6 | 5 |

$h_1 = 2$

$h_2 = 2$

JOS

|   |   |   |
|---|---|---|
| 2 | 8 | 3 |
| 1 |   | 4 |
| 7 | 6 | 5 |

$h_1 = 3$

$h_2 = 4$

DREAPTA

|   |   |   |
|---|---|---|
| 2 | 3 |   |
| 1 | 8 | 4 |
| 7 | 6 | 5 |

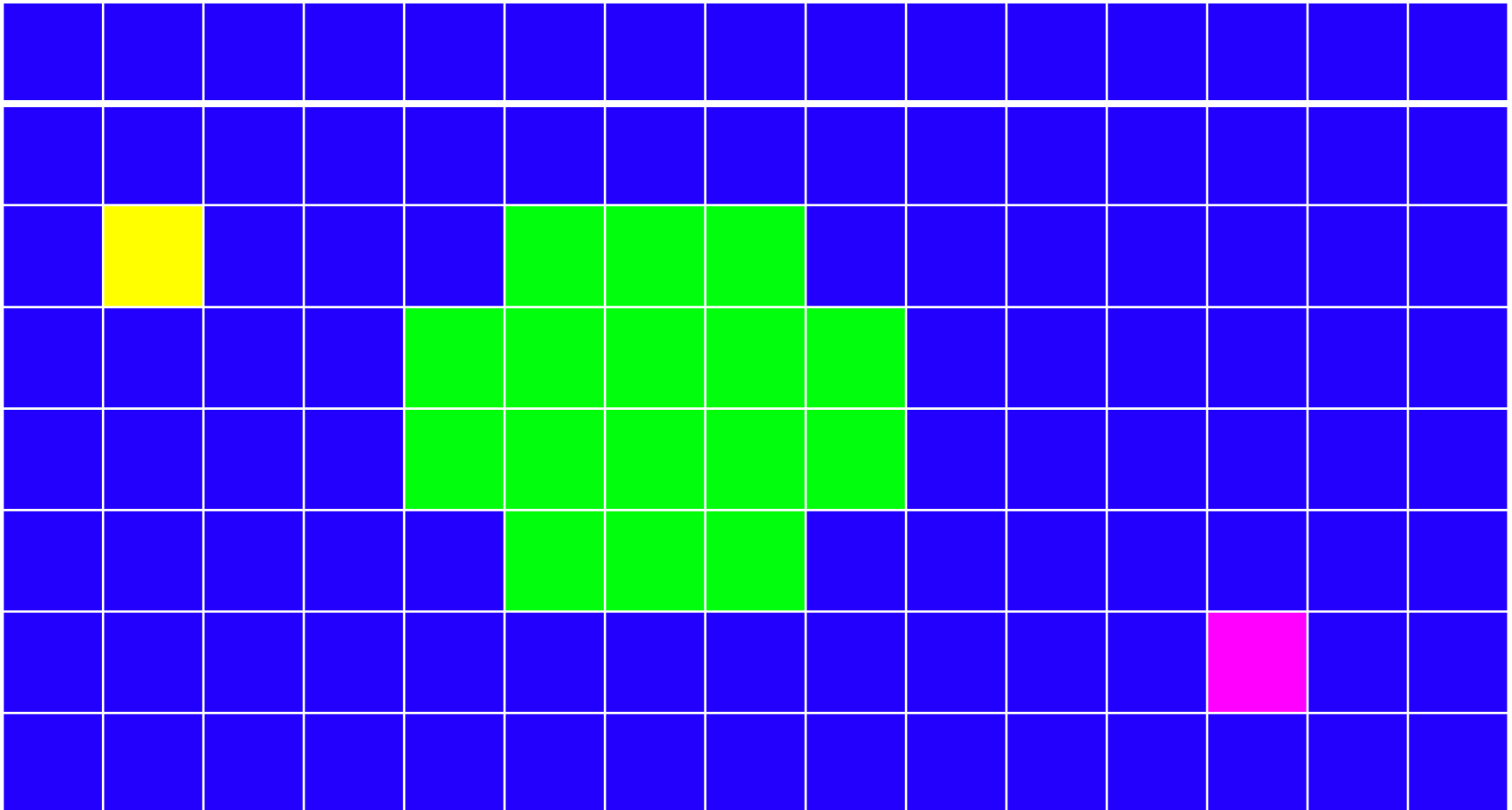
$h_1 = 4$

$h_2 = 4$

# Exemplu de funcție euristică

## Configurație mediu:

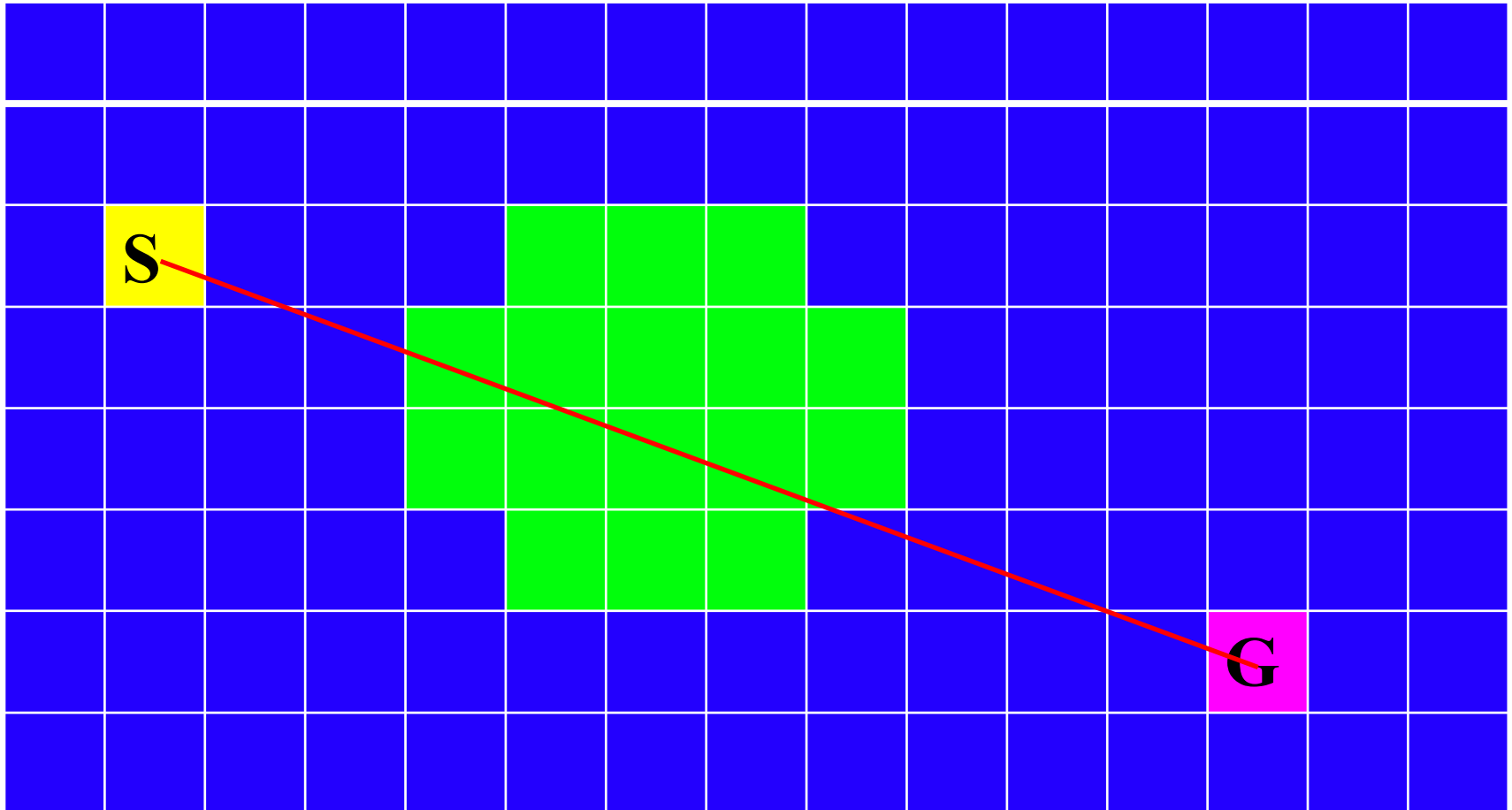
- **stare inițială:** celula marcată cu galben
- **stare scop:** celula marcată cu roz
- **obstacole:** celulele marcate cu verde (nu pot fi traversate)
- **spațiu navigabil:** celulele marcate cu albastru



# Exemplu de funcție euristică

$$h(n) = \text{distanța euclidiană între nodul curent } n \text{ și nodul scop } n_{goal}$$

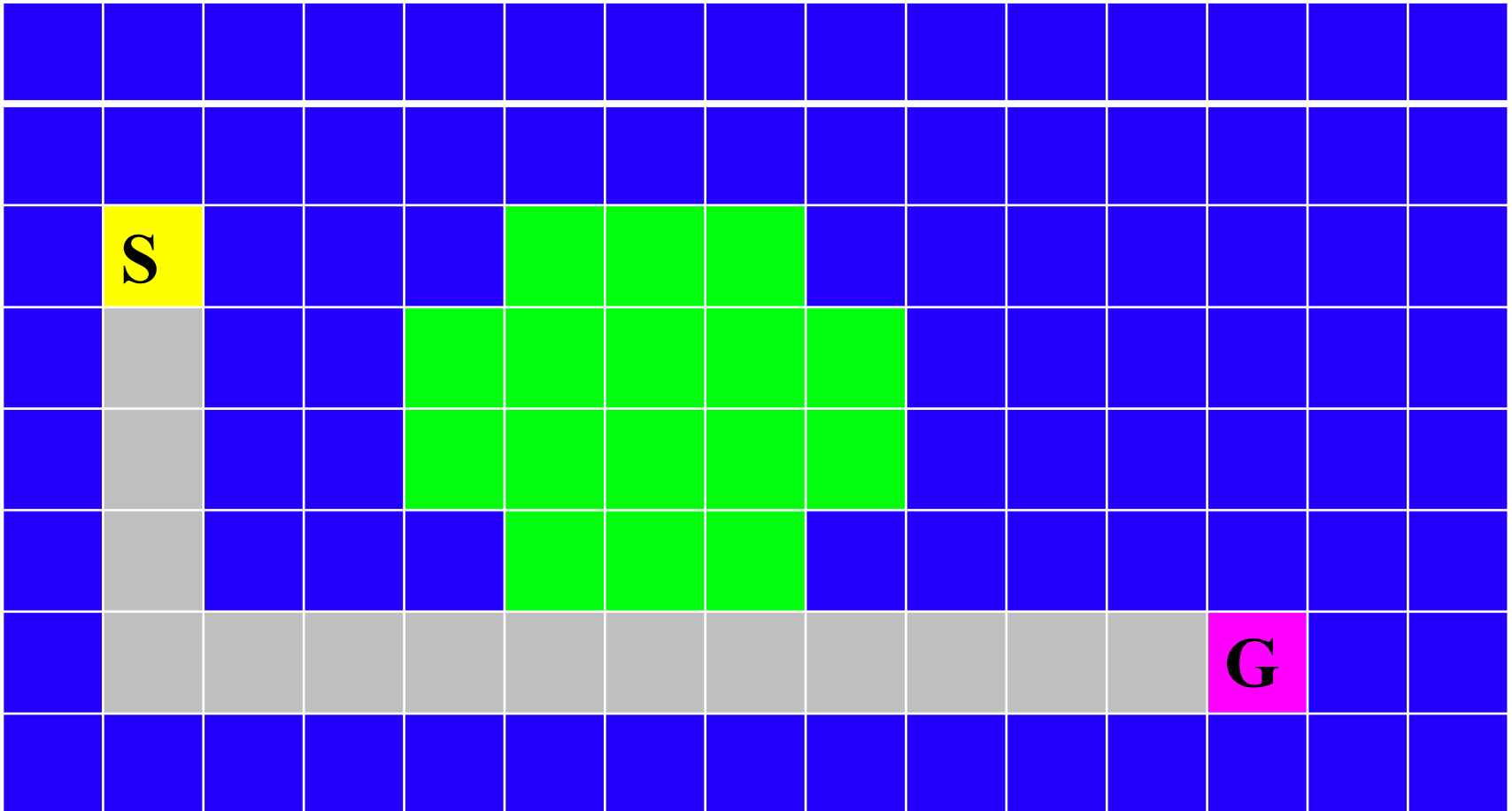
$$h(n) = d_{\text{euclid}}(n, n_{goal}) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$$



# Exemplu de funcție euristică

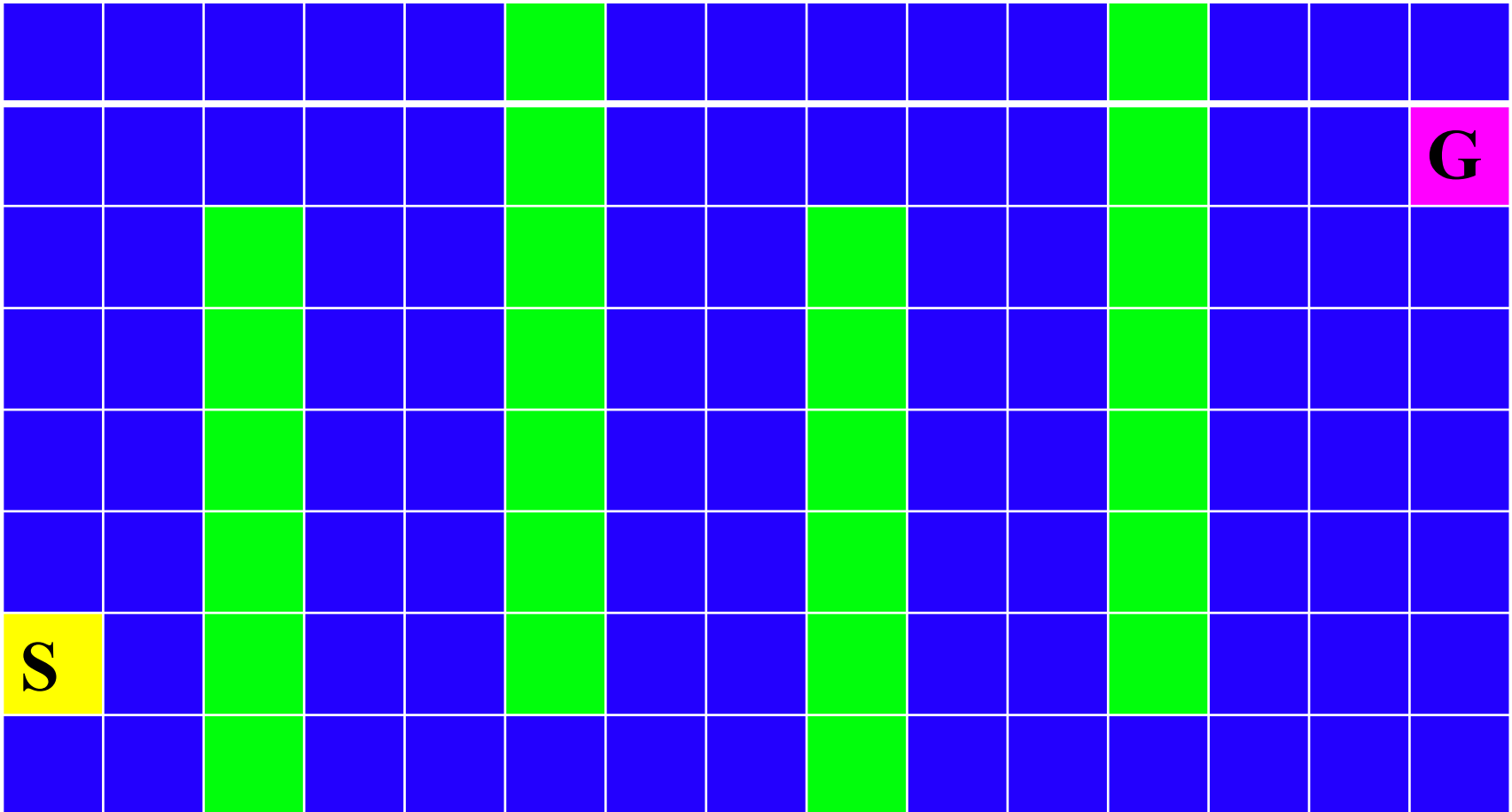
$h(n)$  = distanța Manhattan între nodul curent  $n$  și nodul scop  $n_{goal}$

$$h(n) = d_{\text{Manhattan}}(n, n_{goal}) = |x_n - x_{goal}| + |y_n - y_{goal}|$$



# Exemplu de funcție euristică

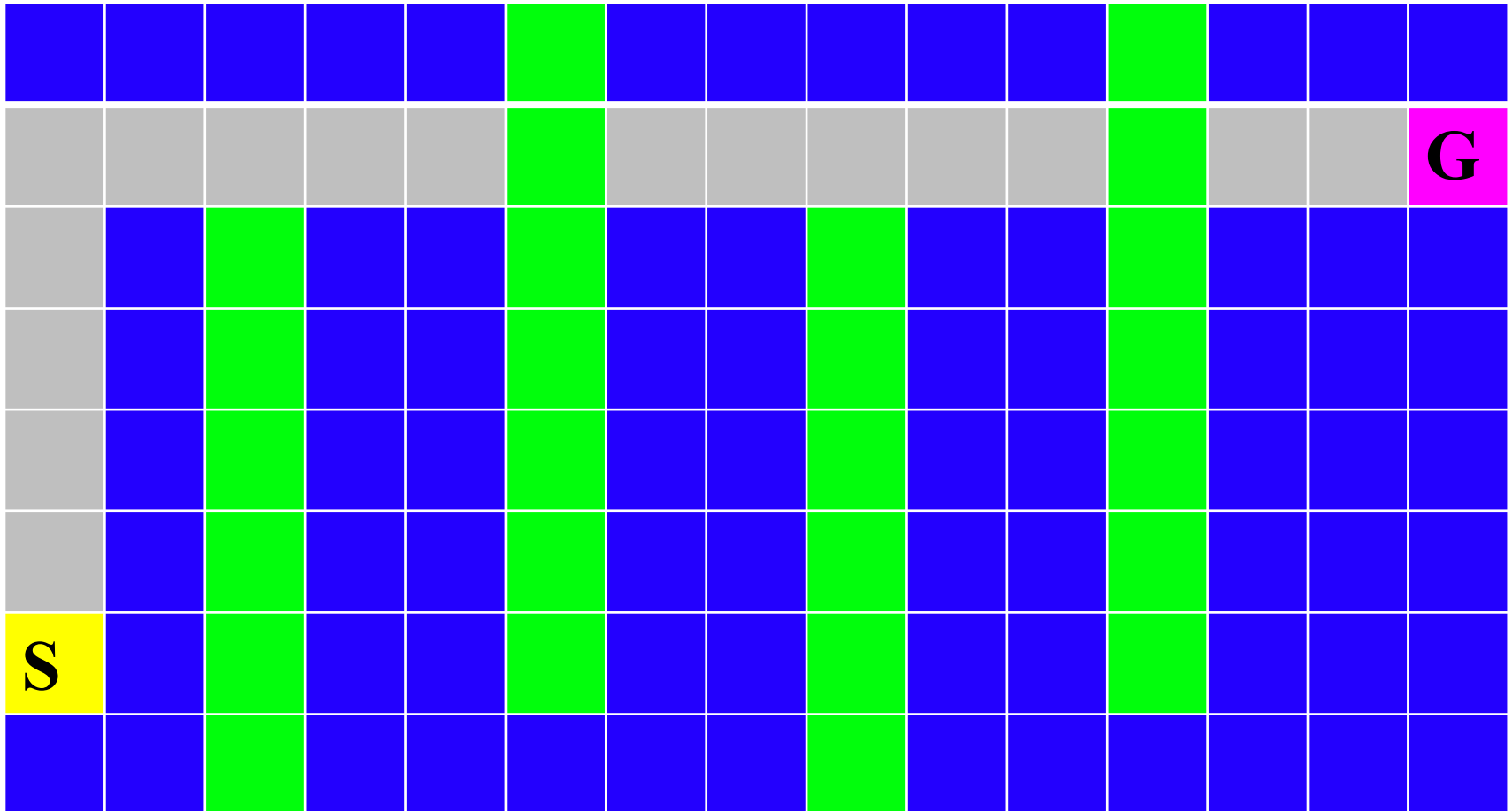
- Distanța Manhattan ignoră obstacolele
- În medii cu bariere → subestimare mare, euristică prea optimistă
- Euristica nu este foarte informativă, nu oferă o margine inferioară strânsă pentru  $h^*(n)$



# Exemplu de funcție euristică

$h(n)$  = distanța Manhattan între nodul curent  $n$  și nodul scop  $n_{goal}$

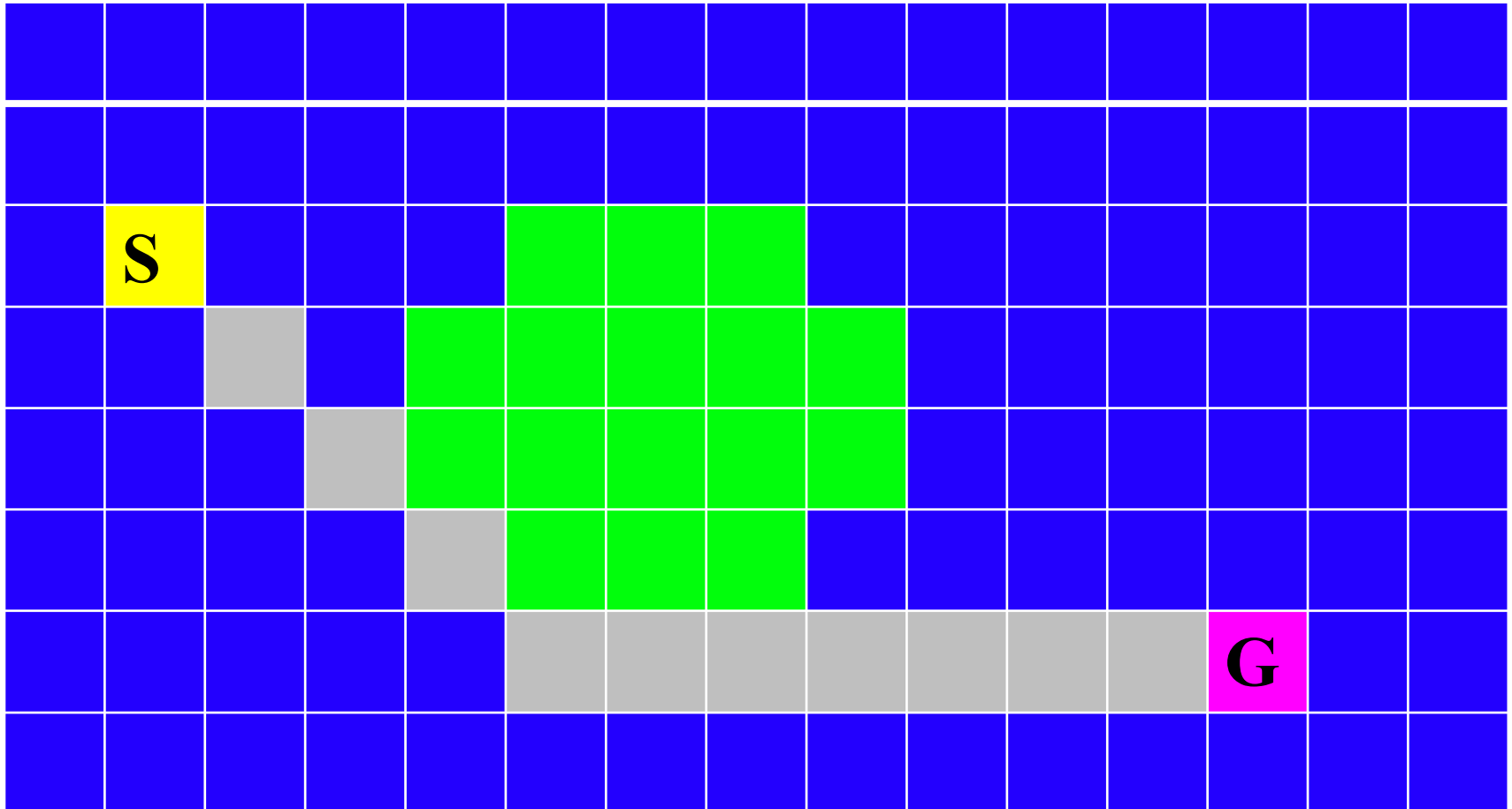
$$h(n) = d_{\text{Manhattan}}(n, n_{goal}) = |x_n - x_{goal}| + |y_n - y_{goal}|$$



# Exemplu de funcție euristică

Într-un grid cu mișcare în 8 direcții și cost unitar, euristica naturală este distanța Chebyshev.

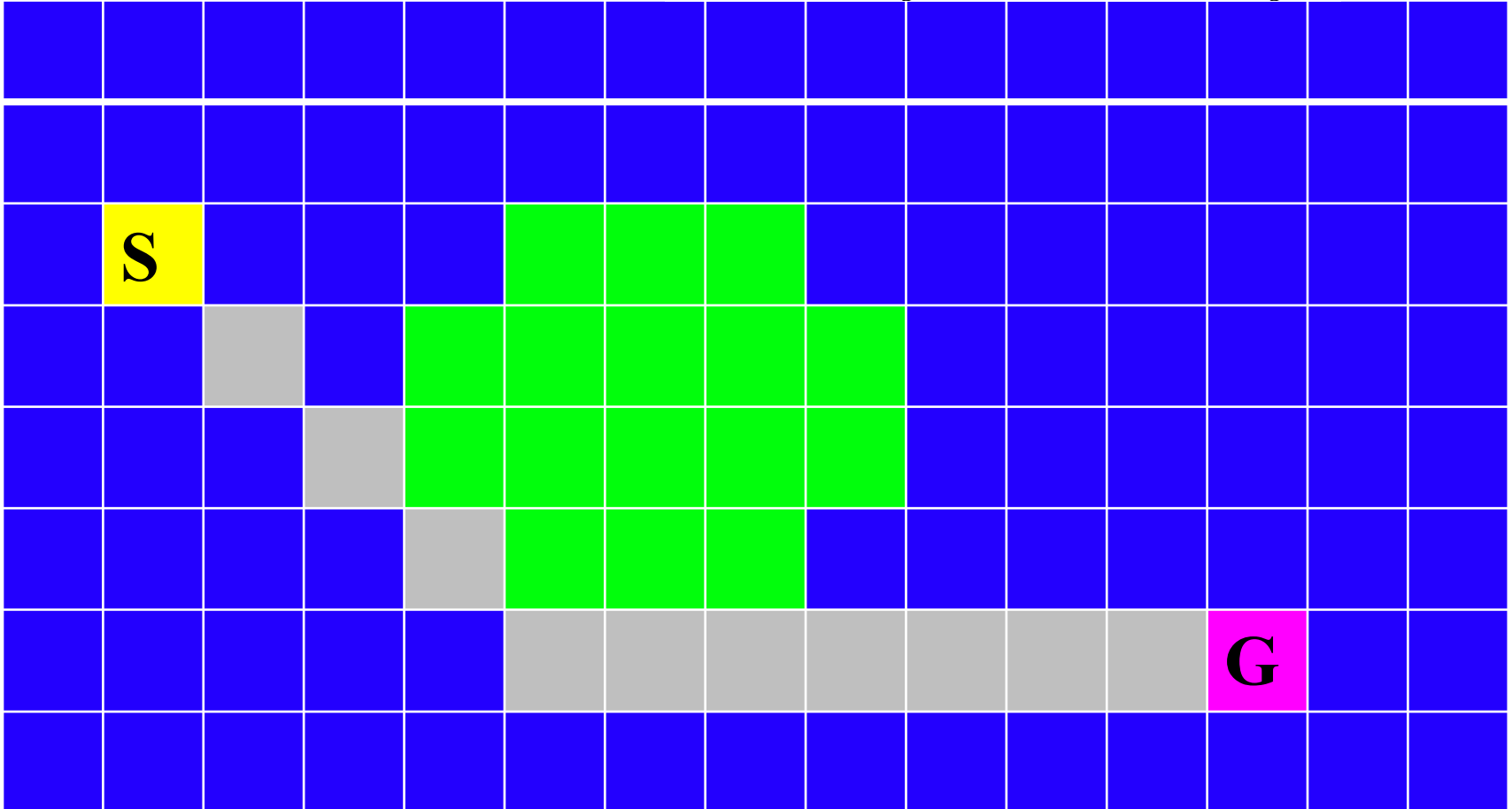
$$h(n) = d_{\text{Chebyshev}}(n, n_{goal}) = \max(|x_n - x_{goal}|, |y_n - y_{goal}|)$$



# Exemplu de funcție euristică

Într-un grid cu mișcare în 8 direcții și costuri neuniforme (1 și  $\sqrt{2}$ ), euristica naturală este distanța Octile.

$$h(n) = d_{\text{Octile}}(n, n_{\text{goal}}) = \max(dx, dy) + (\sqrt{2} - 1) \cdot \min(dx, dy)$$
$$dx = |x_n - x_{\text{goal}}|, \quad dy = |y_n - y_{\text{goal}}|$$





# Căutare Greedy (best-first)



# Căutare Best-First (Best-First Search)

- selectează nodul care va fi expandat pe baza unei funcții de evaluare  $f(n)$
- algoritmul selectează nodul cu valoarea minimă a lui  $f(n)$
- funcția  $f(n)$  estimează cât de promițător este nodul (de obicei distanța până la scop)
- termenul „Best-First” poate fi puțin înșelător:
  - nu știm cu adevărat care este cel mai bun nod
  - algoritmul folosește doar o estimare
  - mai corect ar fi „Best Guess First”

# Căutare Greedy folosind TREE-SEARCH

funcția **GBeFS-TREE-SEARCH** (**problema**, **h**) **returnează** o soluție sau eșec

inițializează **frontiera (Open List)** folosind **starea inițială a problemei**

**ciclează**

**dacă frontiera** este vidă **atunci returnează** eșec

**nod** = **pop\_min\_f(Open List)**

**dacă nodul** conține o **stare scop**

**atunci returnează** soluția corespunzătoare

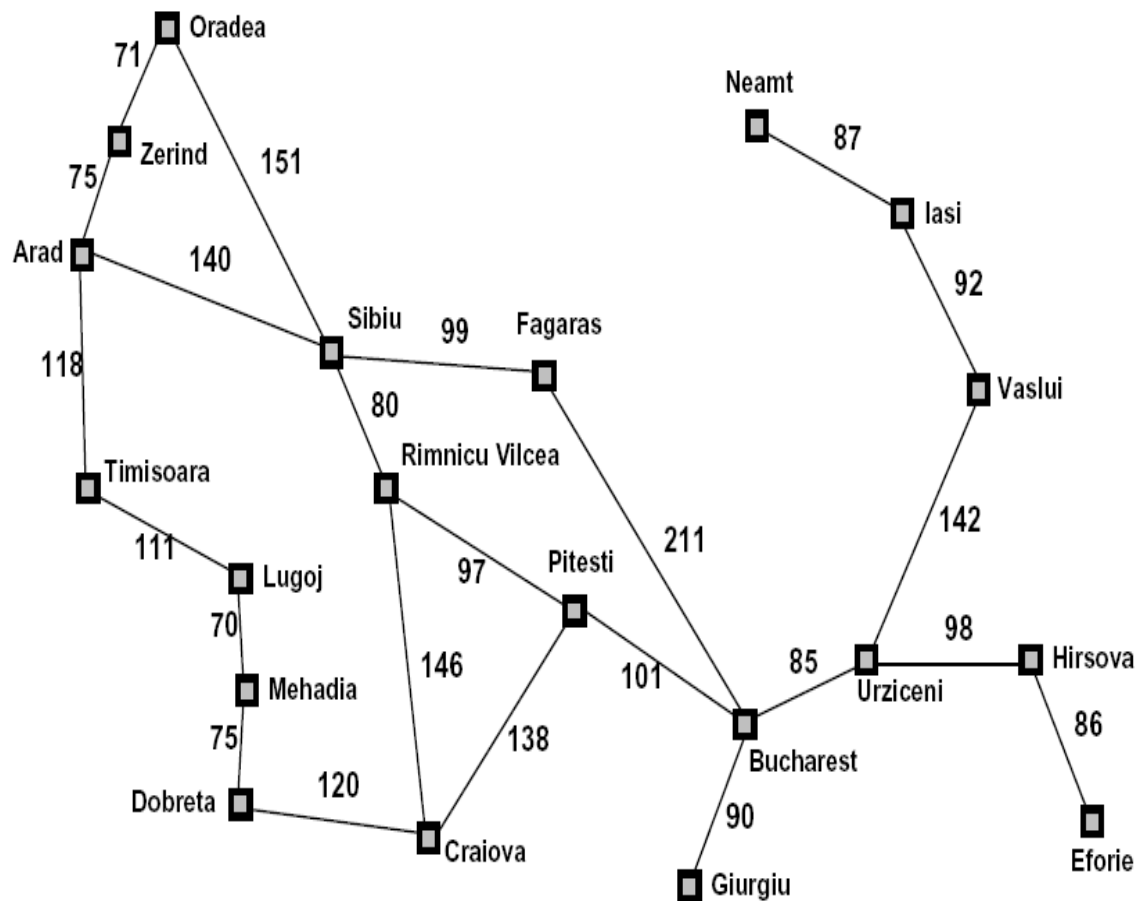
expandează nodul ales, adăugând nodurile succesori generate în **frontieră**

## **Frontiera (OPEN LIST) pentru Greedy Best First Search:**

- structură de date cu noduri generate dar neexpandate
- implementată ca o **coadă de priorități**
- ordonată crescător după  $f(n) = h(n)$
- la fiecare pas se extrage nodul cu **valoarea minimă a euristicii**

# Exemplu de funcție euristică

$h(x)$  = distanța în linie dreaptă între orașul  $x$  și București

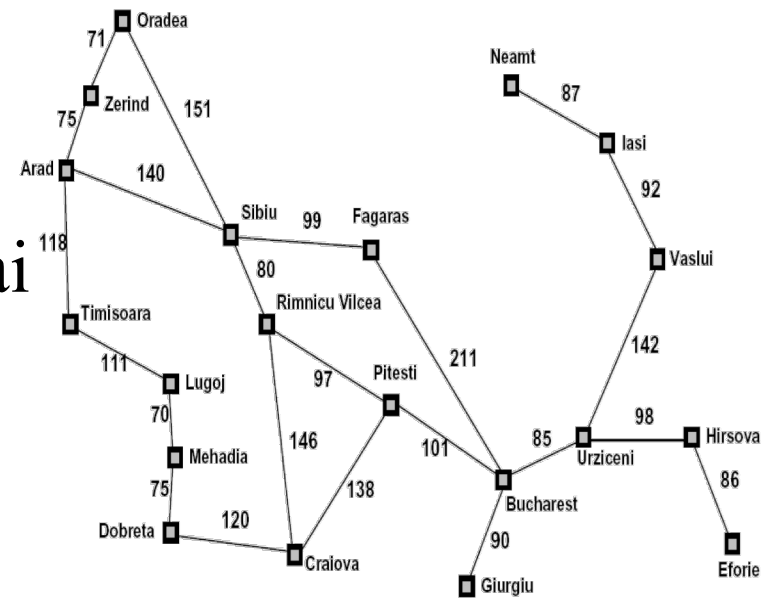


| Straight-line distance to Bucharest |     |
|-------------------------------------|-----|
| Arad                                | 366 |
| Bucharest                           | 0   |
| Craiova                             | 160 |
| Dobreta                             | 242 |
| Eforie                              | 161 |
| Fagaras                             | 178 |
| Giurgiu                             | 77  |
| Hirsova                             | 151 |
| Iasi                                | 226 |
| Lugoj                               | 244 |
| Mehadia                             | 241 |
| Neamt                               | 234 |
| Oradea                              | 380 |
| Pitesti                             | 98  |
| Rimnicu Vilcea                      | 193 |
| Sibiu                               | 253 |
| Timisoara                           | 329 |
| Urziceni                            | 80  |
| Vaslui                              | 199 |
| Zerind                              | 374 |

$h(x)$

# Căutare Greedy

- Explorează nodul care pare cel mai aproape de un nod-scop (dpdv al euristicii, ea ghidează căutarea)

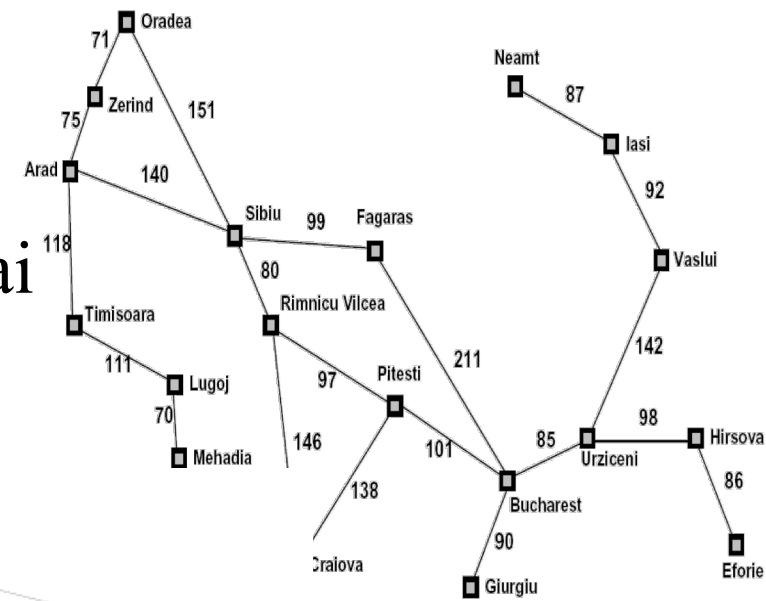
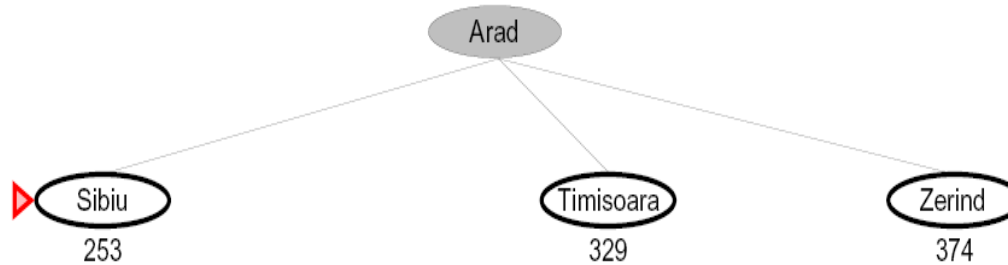


Straight-line distance  
to Bucharest

|                |     |
|----------------|-----|
| Arad           | 366 |
| Bucharest      | 0   |
| Craiova        | 160 |
| Dobreta        | 242 |
| Eforie         | 161 |
| Fagaras        | 178 |
| Giurgiu        | 77  |
| Hirsova        | 151 |
| Iasi           | 226 |
| Lugoj          | 244 |
| Mehadia        | 241 |
| Neamt          | 234 |
| Oradea         | 380 |
| Pitesti        | 98  |
| Rimnicu Vilcea | 193 |
| Sibiu          | 253 |
| Timisoara      | 329 |
| Urziceni       | 80  |
| Vaslui         | 199 |
| Zerind         | 374 |

# Căutare Greedy

- Explorează nodul care pare cel mai aproape de un nod-scop (dpdv al euristicii)

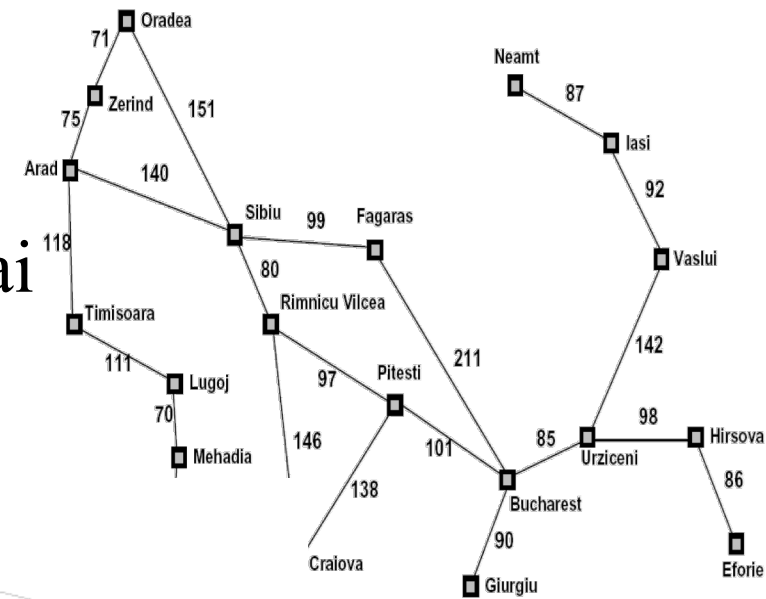
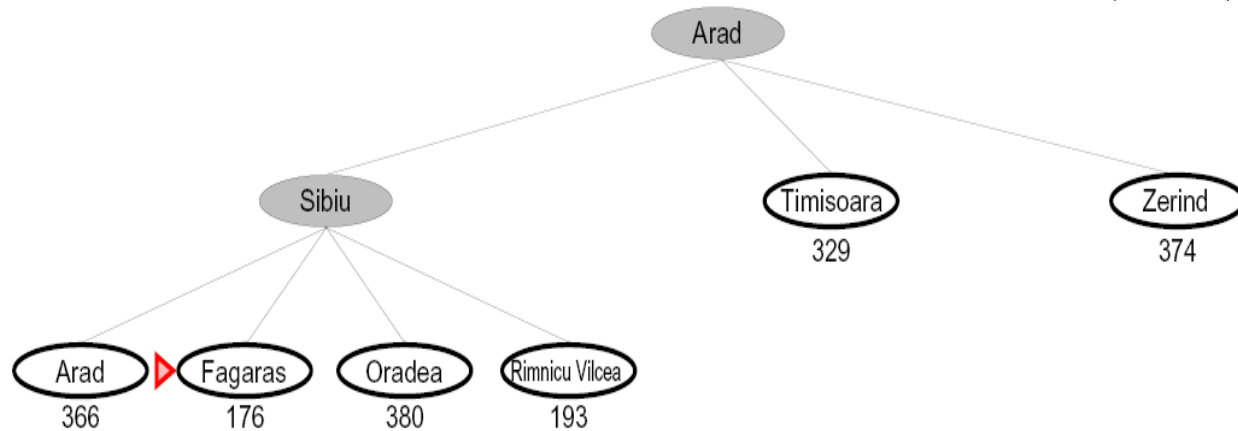


Straight-line distance  
to Bucharest

|                |     |
|----------------|-----|
| Arad           | 366 |
| Bucharest      | 0   |
| Craiova        | 160 |
| Dobreta        | 242 |
| Eforie         | 161 |
| Fagaras        | 178 |
| Giurgiu        | 77  |
| Hirsova        | 151 |
| Iasi           | 226 |
| Lugoj          | 244 |
| Mehadia        | 241 |
| Neamt          | 234 |
| Oradea         | 380 |
| Pitesti        | 98  |
| Rimnicu Vilcea | 193 |
| Sibiu          | 253 |
| Timisoara      | 329 |
| Urziceni       | 80  |
| Vaslui         | 199 |
| Zerind         | 374 |

# Căutare Greedy

- Explorează nodul care pare cel mai aproape de un nod-scop (dpdv al euristicii)

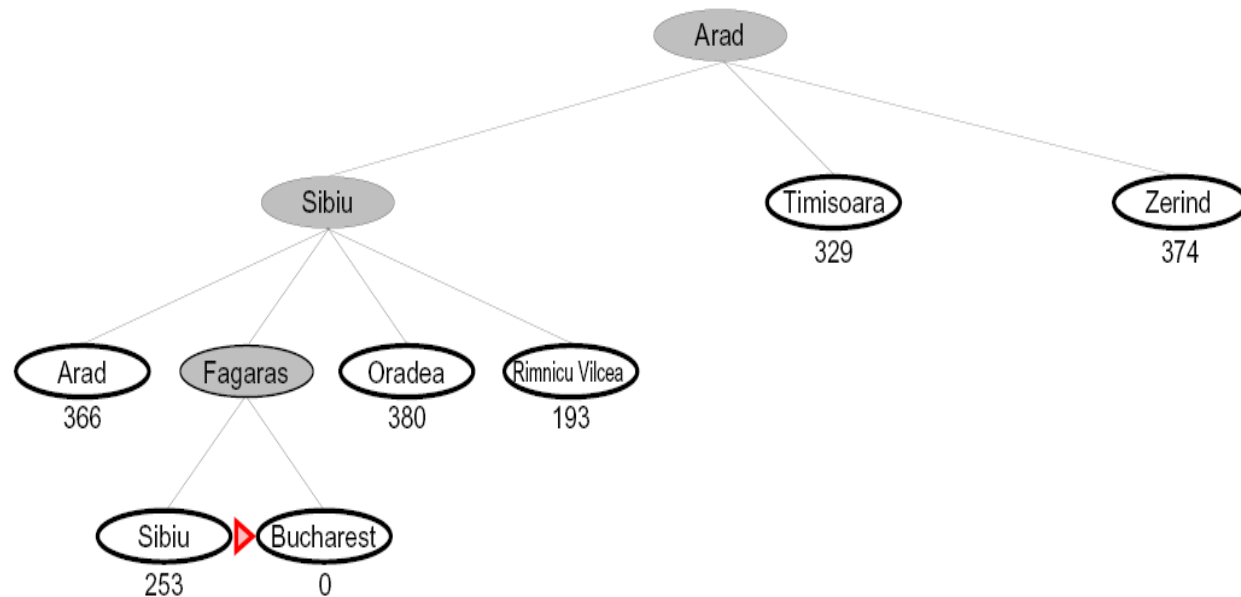


Straight-line distance  
to Bucharest

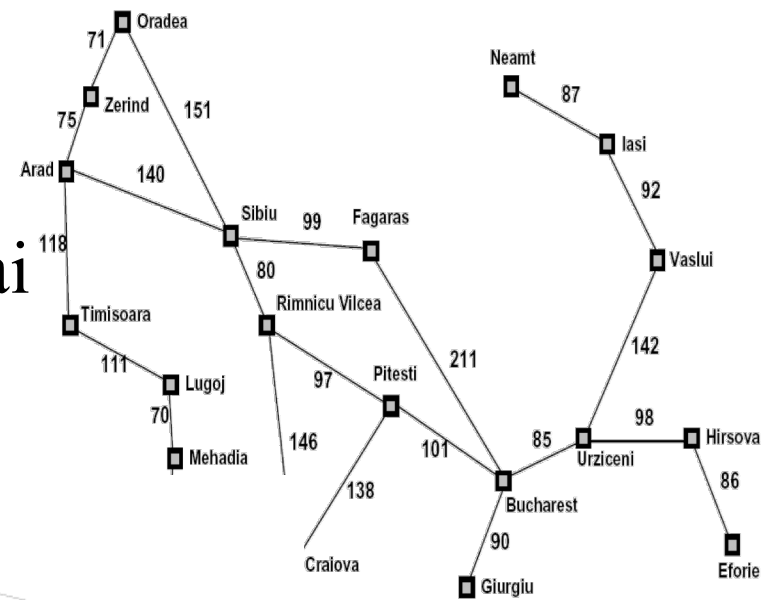
|                |     |
|----------------|-----|
| Arad           | 366 |
| Bucharest      | 0   |
| Craiova        | 160 |
| Dobreta        | 242 |
| Eforie         | 161 |
| Fagaras        | 178 |
| Giurgiu        | 77  |
| Hirsova        | 151 |
| Iasi           | 226 |
| Lugoj          | 244 |
| Mehadia        | 241 |
| Neamt          | 234 |
| Oradea         | 380 |
| Pitesti        | 98  |
| Rimnicu Vilcea | 193 |
| Sibiu          | 253 |
| Timisoara      | 329 |
| Urziceni       | 80  |
| Vaslui         | 199 |
| Zerind         | 374 |

# Căutare Greedy

- Explorează nodul care pare cel mai aproape de un nod-scop (dpdv al euristicii)



- Ce se poate întâmpla?
  - să obținem o soluție suboptimă: Arad – Sibiu – Făgăraș - București, cu un cost mai mare decât soluția optimă: Arad – Sibiu – Râmnicu-Vâlcea-Pitești-București



Straight-line distance to Bucharest

|                |     |
|----------------|-----|
| Arad           | 366 |
| Bucharest      | 0   |
| Craiova        | 160 |
| Dobreta        | 242 |
| Eforie         | 161 |
| Fagaras        | 178 |
| Giurgiu        | 77  |
| Hirsova        | 151 |
| Iasi           | 226 |
| Lugoj          | 244 |
| Mehadia        | 241 |
| Neamt          | 234 |
| Oradea         | 380 |
| Pitesti        | 98  |
| Rimnicu Vilcea | 193 |
| Sibiu          | 253 |
| Timisoara      | 329 |
| Urziceni       | 80  |
| Vaslui         | 199 |
| Zerind         | 374 |